



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

- сообщения на inStudy

ОСНОВЫ ЯЗЫКА СТРУКТУРИРОВАННЫХ ЗАПРОСОВ SQL

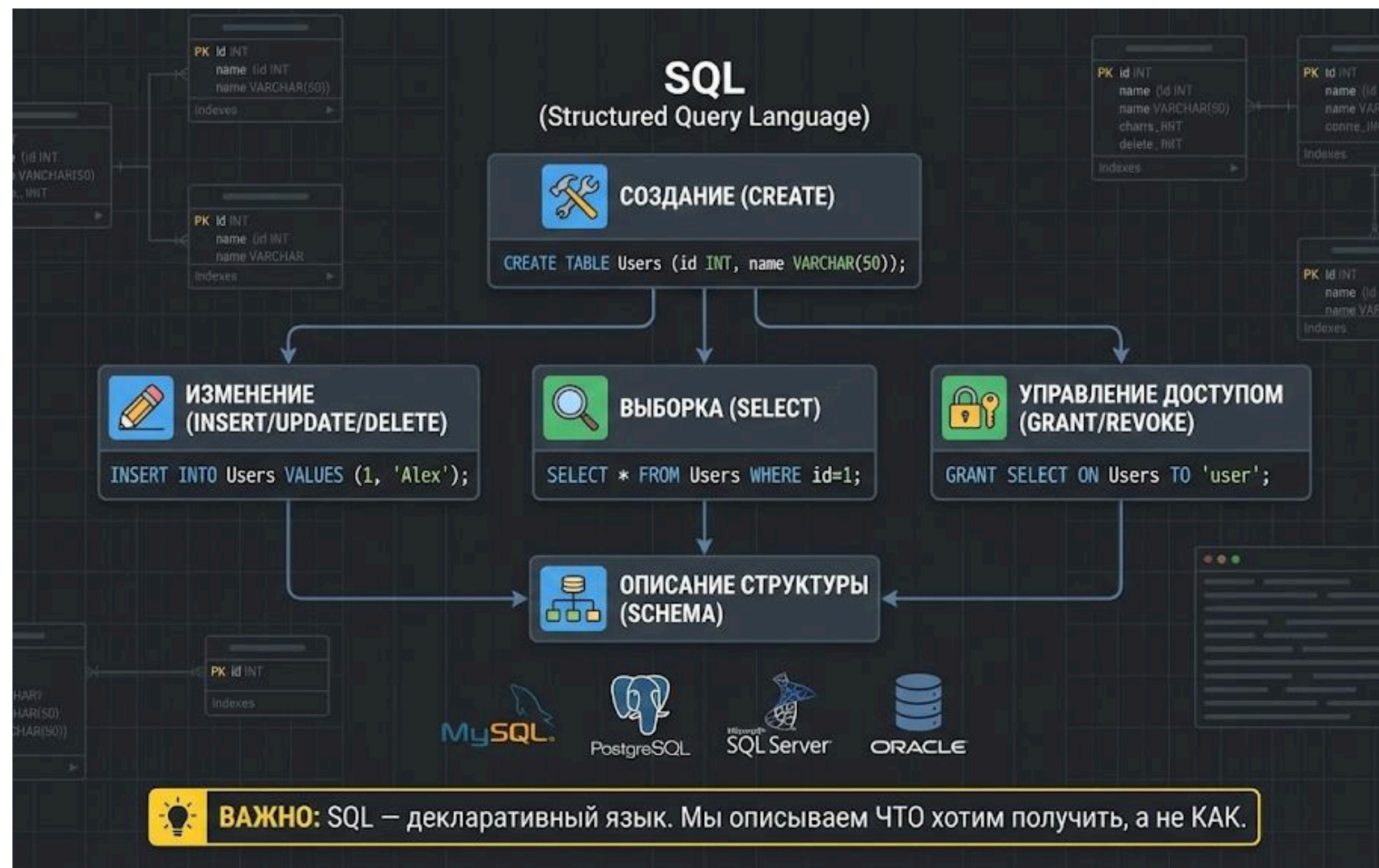


ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

ЧТО ТАКОЕ SQL?

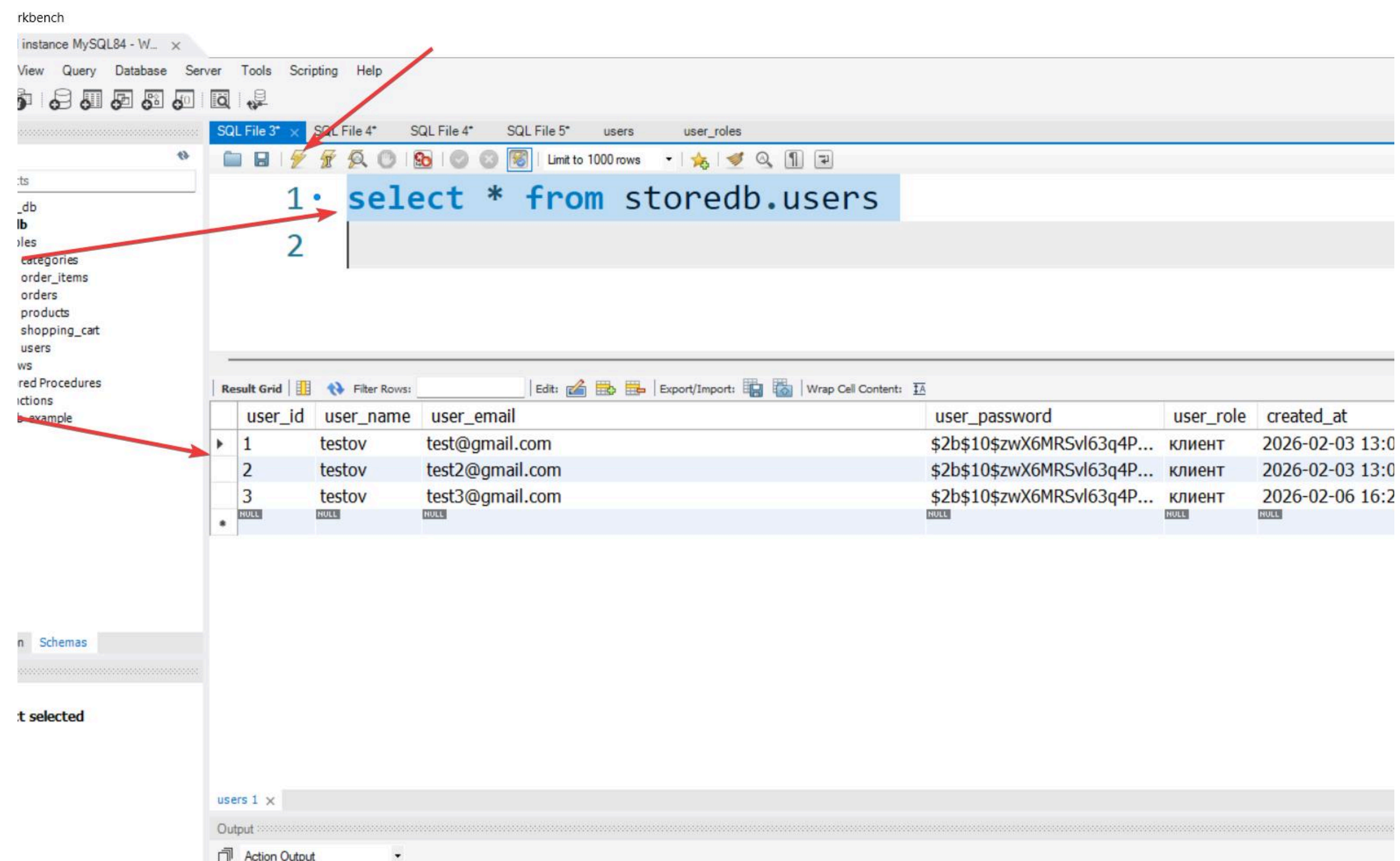
НАЗНАЧЕНИЕ ЯЗЫКА И ЕГО РОЛЬ В РАБОТЕ С БАЗАМИ ДАННЫХ



1. SQL (Structured Query Language) — язык структурированных запросов, предназначенный для работы с реляционными базами данных.
2. С помощью SQL можно:
 - создавать базы данных и таблицы;
 - добавлять, изменять и удалять данные;
 - выполнять выборку данных;
 - управлять доступом;
 - описывать структуру БД.
3. SQL используется в СУБД: MySQL, PostgreSQL, SQL Server, Oracle
4. **Важно:** SQL — декларативный язык. Мы описываем **что хотим получить**, а не как именно это должно быть выполнено.

ГДЕ ВЫПОЛНЯТЬ SQL-ЗАПРОСЫ?

СРЕДЫ И ИНСТРУМЕНТЫ ДЛЯ РАБОТЫ С SQL В MYSQL



1. SQL-запросы выполняются в специальных средах – **SQL-клиентах**. Это программы, которые подключаются к серверу базы данных и позволяют отправлять команды.

2. MySQL Workbench (графическая среда)

3. Как выполнить запрос в MySQL Workbench:

а. Открыть программу

б. Подключиться к серверу (двойной клик по соединению)

с. Нажать SQL Editor

д. Ввести запрос:

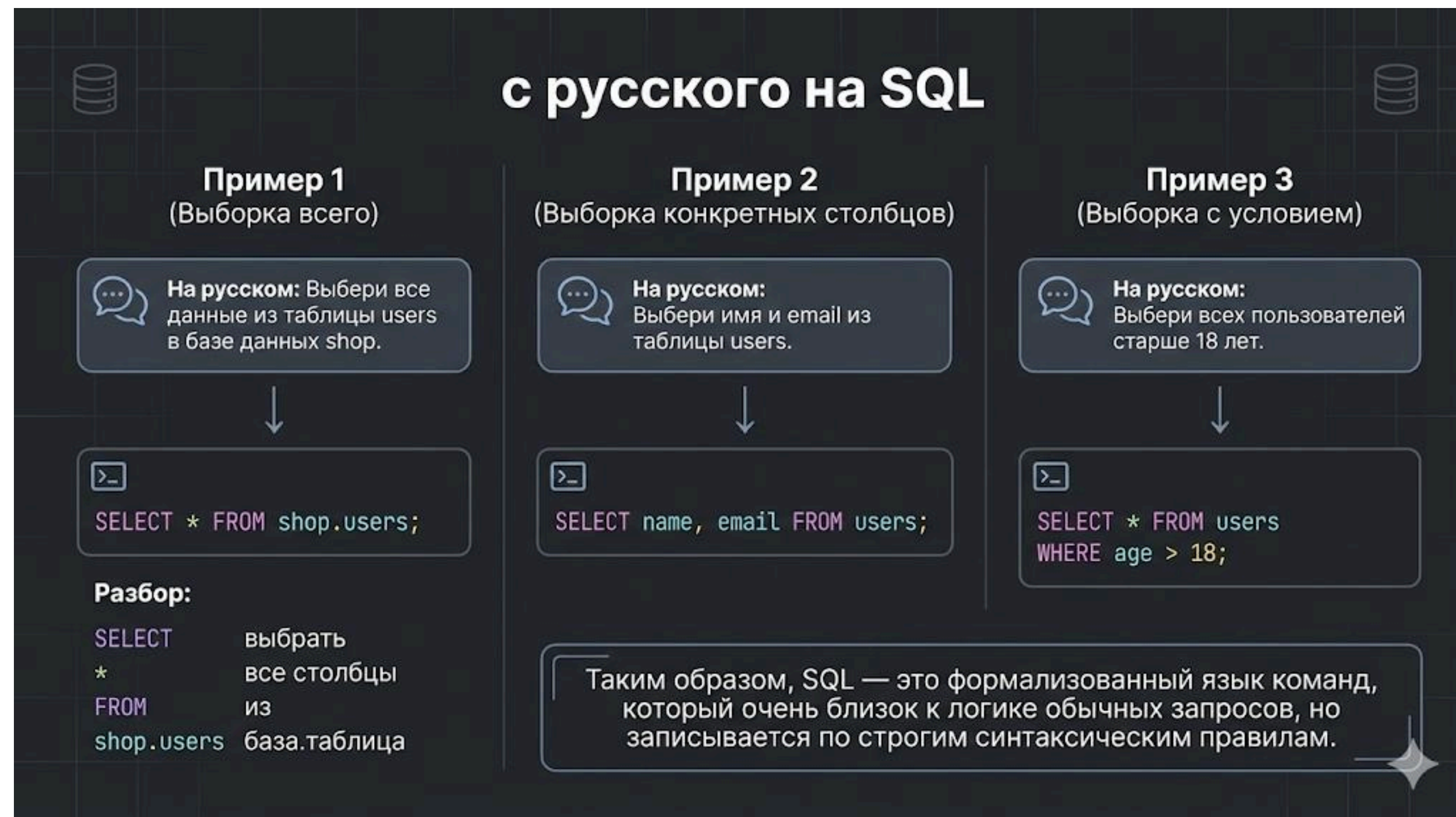
е. `SELECT * FROM users;`

ф. Нажать кнопку ⚡ (Execute)

4. Результат появится в нижней панели.

ПРИМЕР «С РУССКОГО НА SQL»

НАЗНАЧЕНИЕ ЯЗЫКА И ЕГО РОЛЬ В РАБОТЕ С БАЗАМИ ДАННЫХ



1. Представим, что мы формулируем задачу обычным языком.

2. **На русском:**

а. *Выбери все данные из таблицы users в базе данных shop.*

3. **На SQL:**

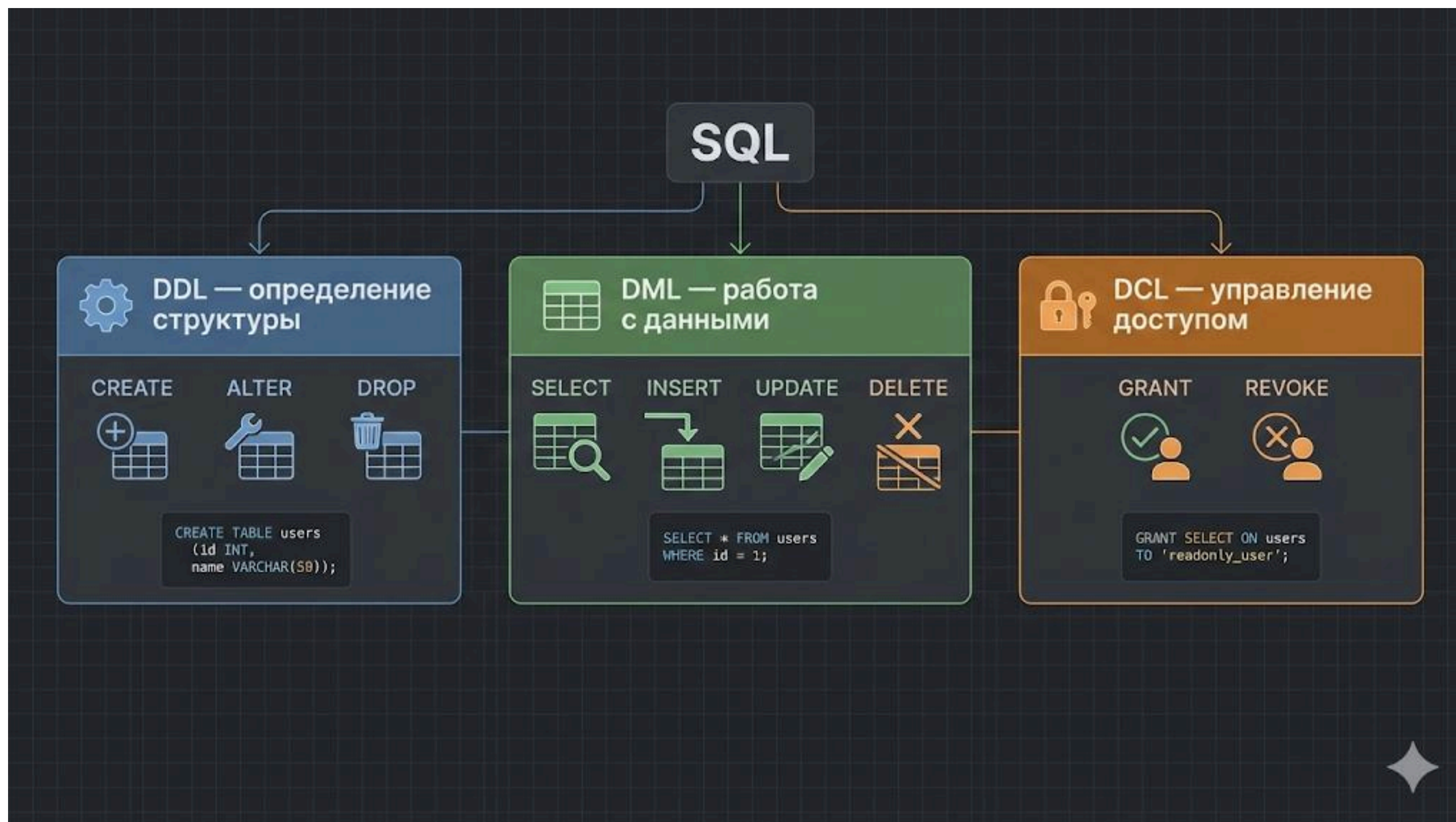
а. *SELECT * FROM shop.users;*

4. **Разбор:**

- SELECT — выбрать
- * — все столбцы
- FROM — из
- shop.users — база данных и таблица

КАТЕГОРИИ КОМАНД SQL

КЛАССИФИКАЦИЯ SQL ПО ТИПУ ВЫПОЛНЯЕМЫХ ОПЕРАЦИЙ



1. SQL делится на группы:

2. DDL — **определение структуры**

- CREATE - создание таблиц
- ALTER - изменение таблиц
- DROP - удаление таблиц

3. DML — **работа с данными**

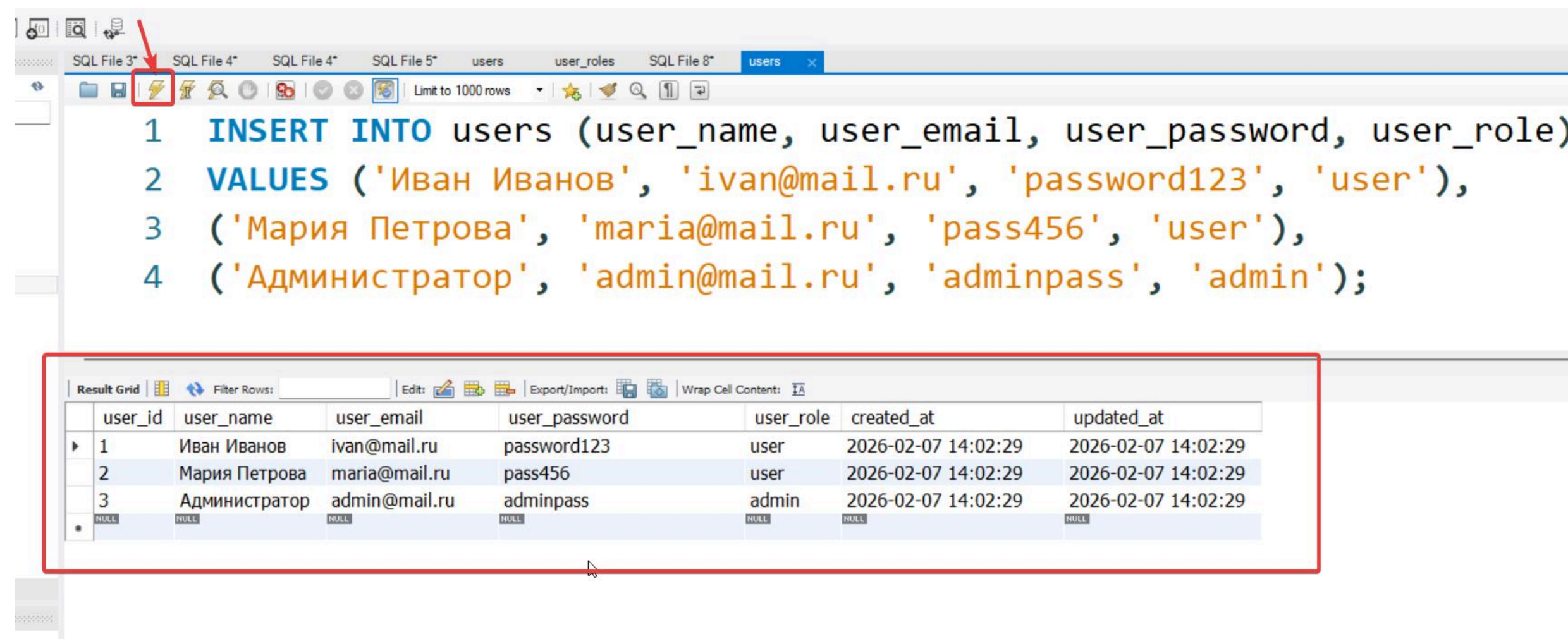
- SELECT - поиск/выборка данных
- INSERT - вставка строки таблицы
- UPDATE - обновление строки таблицы
- DELETE - удаление строки из таблицы

4. DCL — **управление доступом**

- GRANT - выдать разрешение
- REVOKE -

КОМАНДА INSERT В MYSQL

ДОБАВЛЕНИЕ ДАННЫХ В ТАБЛИЦУ



1. INSERT – это DML-команда (Data Manipulation Language), предназначенная для добавления новых строк в таблицу базы данных.
2. Используется для:
 - заполнения таблиц начальными данными
 - добавления новых записей (пользователь, заказ, товар и т.д.)
 - массовой загрузки данных
3. Общий принцип:
 - a. **INSERT INTO table_name (column1, column2, ...)**
 - b. **VALUES (value1, value2, ...);**

БАЗОВЫЙ СИНТАКСИС INSERT

ЯВНОЕ УКАЗАНИЕ СТОЛБЦОВ

```
1 INSERT INTO products (product_name, product_price, product_description, product_img, category_id, stock, is_active)
2 VALUES
3 ('Смартфон Samsung Galaxy S21', 79999.99, 'Флагманский смартфон Samsung с 8GB RAM', 'img/s21.jpg', 1, 10, TRUE),
4 ('Ноутбук Lenovo ThinkPad', 120000.00, 'Ноутбук для работы и учебы', 'img/thinkpad.jpg', 1, 5, TRUE),
5 ('Футболка Adidas', 1999.50, 'Спортивная футболка для повседневной носки', 'img/adidas_tshirt.jpg', 2, 50, TRUE),
6 ('Книга "Война и мир"', 599.00, 'Роман Л. Н. Толстого', 'img/war_and_peace.jpg', 3, 30, TRUE),
7 ('Конструктор LEGO', 4999.00, 'Конструктор для детей от 6 лет', 'img/lego.jpg', 4, 20, TRUE),
8 ('Кофейная кружка', 399.00, 'Керамическая кружка 350мл', 'img/mug.jpg', 5, 100, TRUE);
```

Result Grid

	product_id	product_name	product_price	product_description	product_img	category_id	stock	is_active	created_at	updated_at
1		Смартфон Samsung Galaxy S21	79999.99	Флагманский смартфон Samsu...	img/s21.jpg	1	10	1	2026-02-07 14:02:29	2026-02-07 14:02:29
2		Ноутбук Lenovo ThinkPad	120000.00	Ноутбук для работы и учебы	img/thinkpad.jpg	1	5	1	2026-02-07 14:02:29	2026-02-07 14:02:29
3		Футболка Adidas	1999.50	Спортивная футболка для пов...	img/adidas_tshirt.jpg	2	50	1	2026-02-07 14:02:29	2026-02-07 14:02:29
4		Книга "Война и мир"	599.00	Роман Л. Н. Толстого	img/war_and_peace.jpg	3	30	1	2026-02-07 14:02:29	2026-02-07 14:02:29
5		Конструктор LEGO	4999.00	Конструктор для детей от 6 лет	img/lego.jpg	4	20	1	2026-02-07 14:02:29	2026-02-07 14:02:29
6		Кофейная кружка	399.00	Керамическая кружка 350мл	img/mug.jpg	5	100	1	2026-02-07 14:02:29	2026-02-07 14:02:29

1. Общий принцип:
- а. INSERT INTO users (name, email, age)

б. VALUES ('Alex', 'alex@mail.com', 25);
2. Что происходит:
- **users** — имя таблицы
 - **(name, email, age)** — список столбцов
 - **VALUES** — значения в том же порядке
 - Каждое значение соответствует своему столбцу
3. Важно:
- Количество столбцов = количеству значений
 - Тип данных должен соответствовать типу столбца

AUTO_INCREMENT И INSERT

АВТОМАТИЧЕСКАЯ ГЕНЕРАЦИЯ ИДЕНТИФИКАТОРА

Limit to 1000 rows

1

INSERT INTO products (product_name, product_price, product_description, product_img, category_id, stock, is_active)

2

VALUES

3

('Смартфон Samsung Galaxy S21', 79999.99, 'Флагманский смартфон Samsung с 8GB RAM', 'img/s21.jpg', 1, 10, TRUE),

4

('Ноутбук Lenovo ThinkPad', 120000.00, 'Ноутбук для работы и учебы', 'img/thinkpad.jpg', 1, 5, TRUE),

5

('Футболка Adidas', 1999.50, 'Спортивная футболка для повседневной носки', 'img/adidas_tshirt.jpg', 2, 50, TRUE),

6

('Книга "Война и мир"', 599.00, 'Роман Л. Н. Толстого', 'img/war_and_peace.jpg', 3, 30, TRUE),

7

('Конструктор LEGO', 4999.00, 'Конструктор для детей от 6 лет', 'img/lego.jpg', 4, 20, TRUE),

8

('Кофейная кружка', 399.00, 'Керамическая кружка 350мл', 'img/mug.jpg', 5, 100, TRUE);

Result Grid

Filter Rows:

Edit

Export/Imports

Wrap Cell Contents

	product_id	product_name	product_price	product_description	product_img	category_id	stock	is_active	created_at	updated_at
	1	Смартфон Samsung Galaxy S21	79999.99	Флагманский смартфон Samsu...	img/s21.jpg	1	10	1	2026-02-07 14:02:29	2026-02-07 14:02:29
	2	Ноутбук Lenovo ThinkPad	120000.00	Ноутбук для работы и учебы	img/thinkpad.jpg	1	5	1	2026-02-07 14:02:29	2026-02-07 14:02:29
	3	Футболка Adidas	1999.50	Спортивная футболка для пов...	img/adidas_tshirt.jpg	2	50	1	2026-02-07 14:02:29	2026-02-07 14:02:29
	4	Книга "Война и мир"	599.00	Роман Л. Н. Толстого	img/war_and_peace.jpg	3	30	1	2026-02-07 14:02:29	2026-02-07 14:02:29
	5	Конструктор LEGO	4999.00	Конструктор для детей от 6 лет	img/lego.jpg	4	20	1	2026-02-07 14:02:29	2026-02-07 14:02:29
	6	Кофейная кружка	399.00	Керамическая кружка 350мл	img/mug.jpg	5	100	1	2026-02-07 14:02:29	2026-02-07 14:02:29

»

1. Если столбец объявлен как:
- a. id INT PRIMARY KEY **AUTO_INCREMENT**
2. То при вставке его можно не указывать:
- a. INSERT INTO users (name, email, age)
- b. VALUES ('Alex', 'alex@mail.com', 25);
3. MySQL автоматически присвоит **id**.

INSERT И NULL

РАБОТА С ПУСТЫМИ ЗНАЧЕНИЯМИ



The screenshot shows a database client window with a toolbar at the top. The SQL editor contains two lines of code: `1 • INSERT INTO storedb.users (user_name, user_email, user_password, user_role)` and `2 VALUES ('Иван Иванов 2', null, 'password123', 'user');`. Below the editor is an 'Output' panel with a dropdown menu set to 'Action Output'. It displays a table with four columns: '#', 'Time', 'Action', and 'Message'. The first row shows an error: a red 'x' icon, the number '1', the time '10:31:16', the full SQL statement, and the message 'Error Code: 1048. Column 'user_email' cannot be null'.

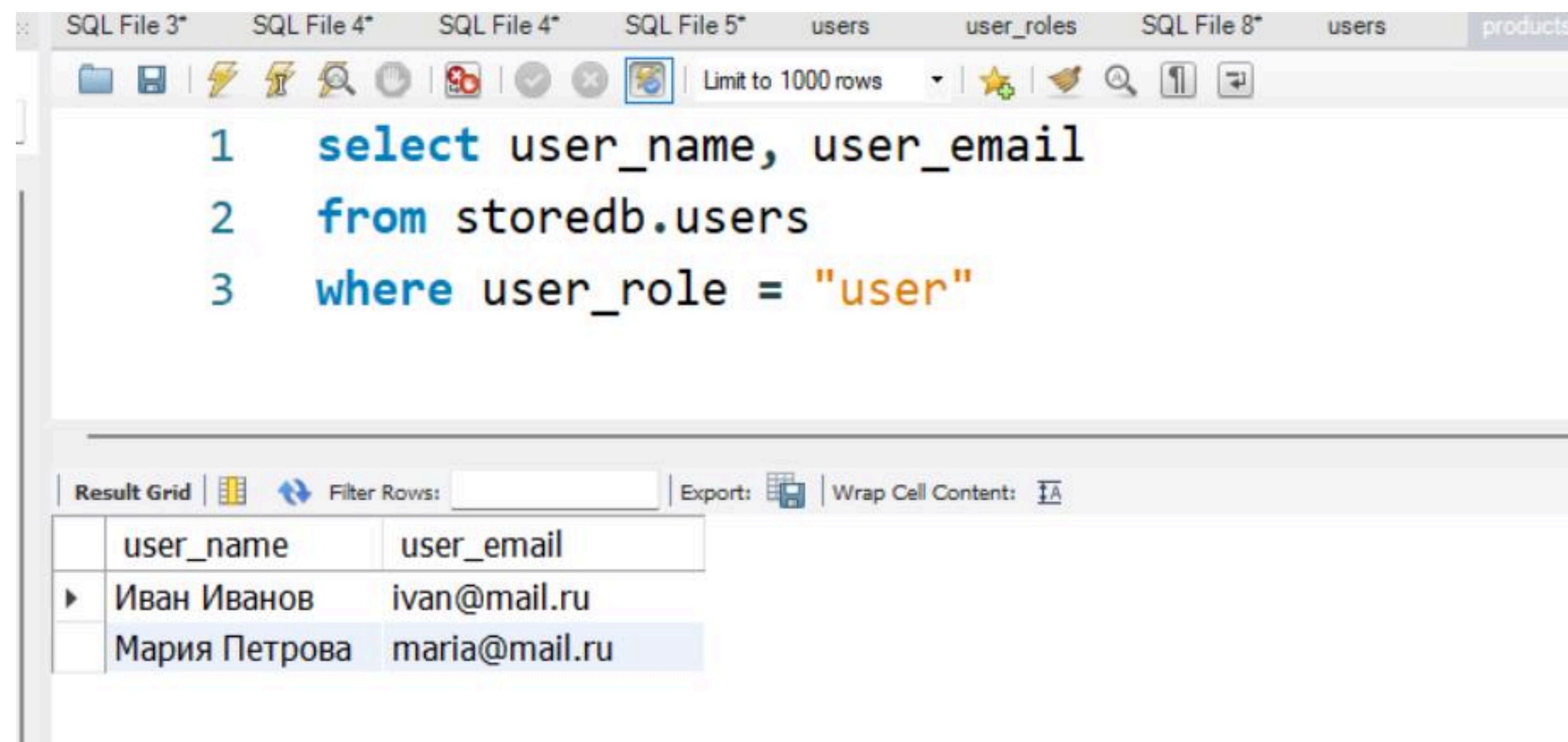
```
1 • INSERT INTO storedb.users (user_name, user_email, user_password, user_role)
2 VALUES ('Иван Иванов 2', null, 'password123', 'user');
```

#	Time	Action	Message
1	10:31:16	INSERT INTO storedb.users (user_name, user_email, user_password, user_role) VALUES ('Иван Иванов 2', null, 'password123', 'user')	Error Code: 1048. Column 'user_email' cannot be null

1. Если столбец **допускает** NULL:
 - a. INSERT INTO users (name, email, age)
 - b. VALUES ('Alex', **NULL**, 25);
2. Если столбец объявлен как:
 - a. email VARCHAR(100) **NOT NULL**
 - b. **Попытка вставить NULL вызовет ошибку.**

КОМАНДА SELECT

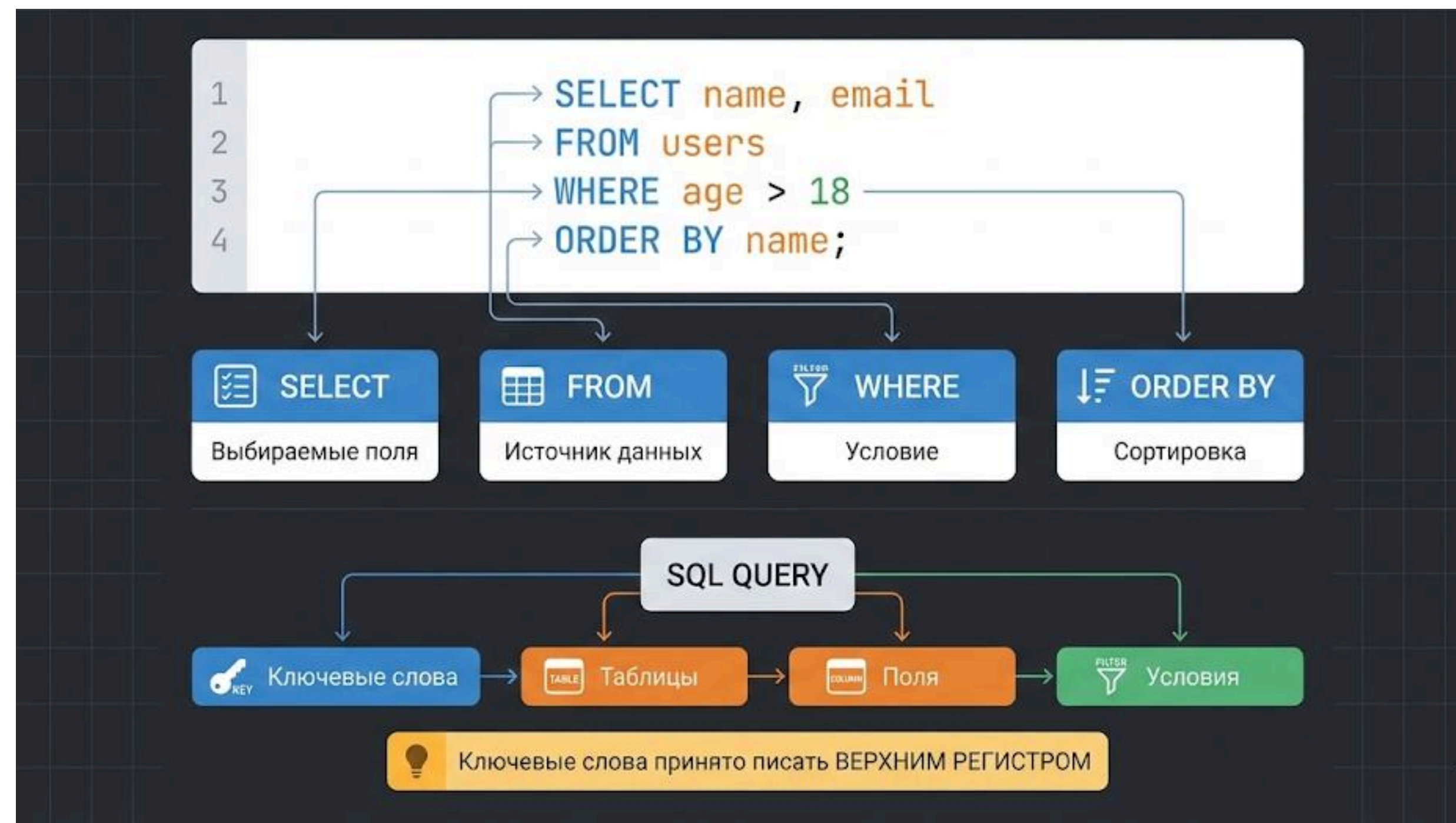
ОСНОВНАЯ КОМАНДА ДЛЯ ПОЛУЧЕНИЯ ДАННЫХ ИЗ БАЗЫ ДАННЫХ



1. SELECT – это команда SQL для выборки данных из таблиц.
2. С её помощью можно:
 - получать все данные
 - выбирать отдельные столбцы
 - фильтровать строки
 - сортировать результат
 - объединять таблицы
 - агрегировать данные

СТРУКТУРА SQL-ЗАПРОСА

ОСНОВНЫЕ ЭЛЕМЕНТЫ СИНТАКСИСА SQL



1. Разбор:

- **SELECT** — выбираемые поля
- **FROM** — источник данных
- **WHERE** — условие
- **ORDER BY** — сортировка

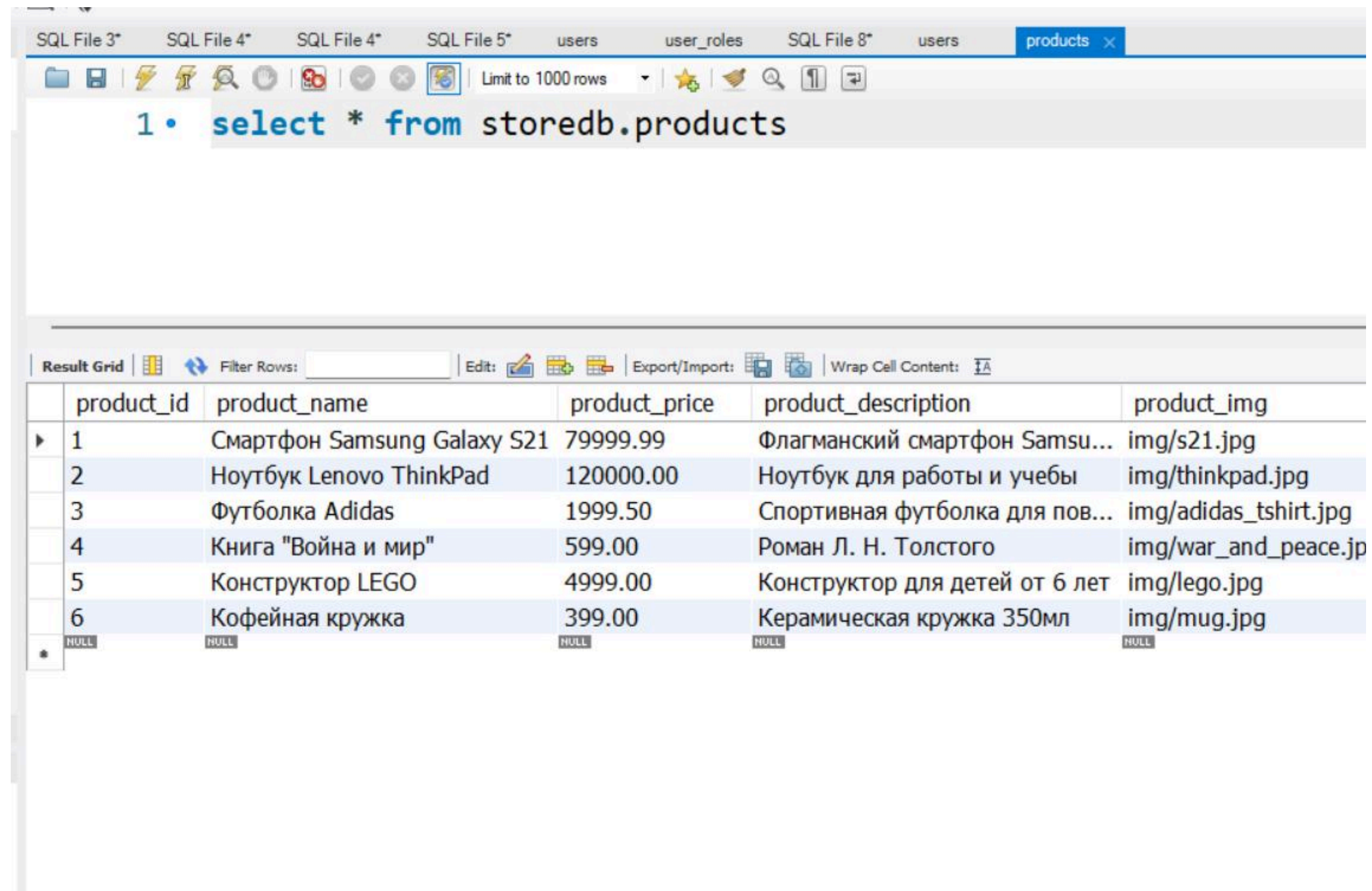
2. Принцип построения:

- a. SELECT [DISTINCT]** список_столбцов или *
- FROM** источник
- WHERE** фильтр
- ORDER BY** выражение_сортировки

3. Ключевые слова принято писать ВЕРХНИМ РЕГИСТРОМ.

ВЫБОР ВСЕХ СТОЛБЦОВ

ИСПОЛЬЗОВАНИЕ СИМВОЛА *



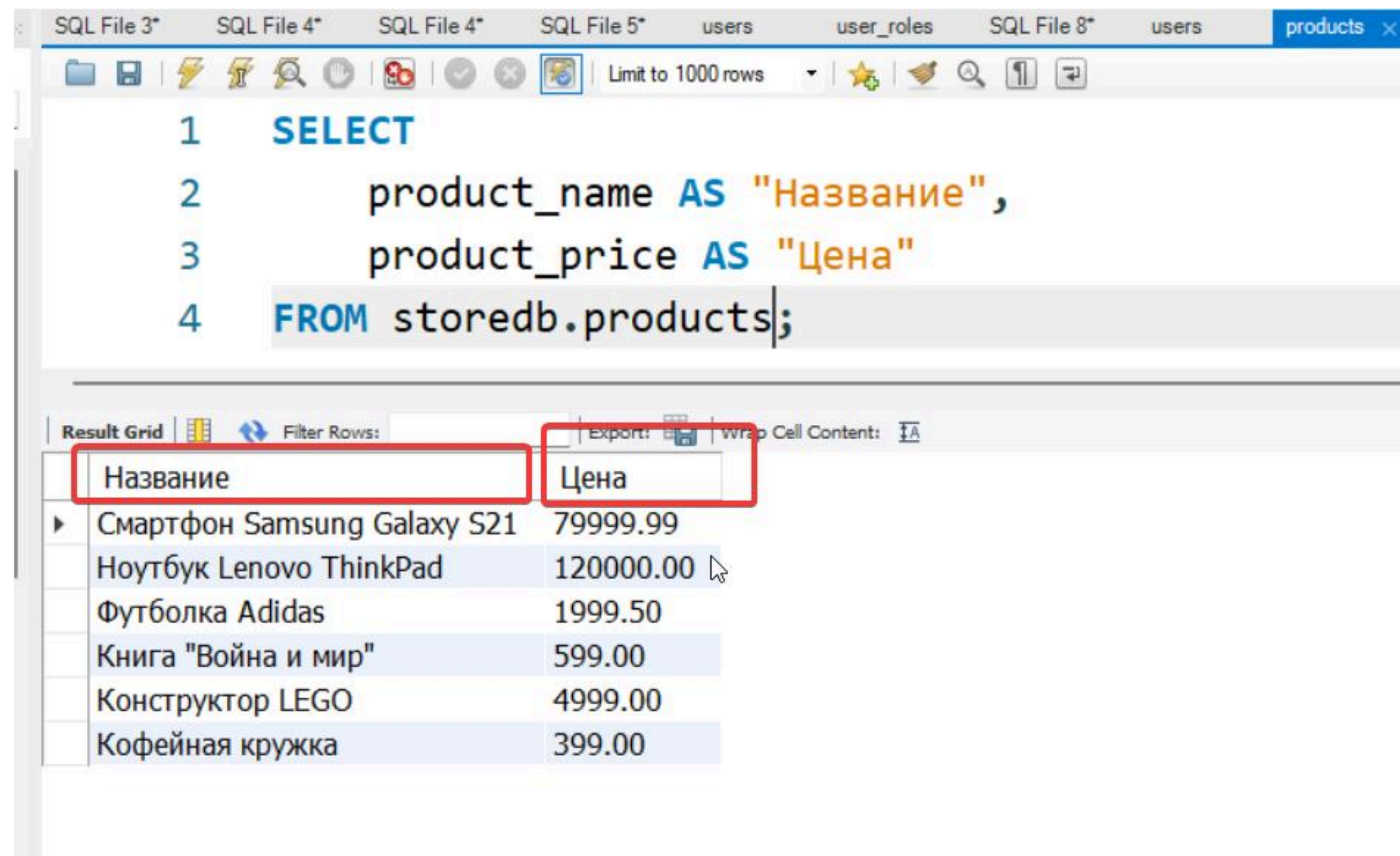
The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and search. The query editor displays the SQL statement: `1 • select * from storedb.products`. Below the editor, the 'Result Grid' shows the output of the query. The grid has columns for product_id, product_name, product_price, product_description, and product_img. It contains 6 rows of product data and a final row with NULL values. The interface also shows a 'Limit to 1000 rows' dropdown and various utility icons.

	product_id	product_name	product_price	product_description	product_img
▶	1	Смартфон Samsung Galaxy S21	79999.99	Флагманский смартфон Samsu...	img/s21.jpg
	2	Ноутбук Lenovo ThinkPad	120000.00	Ноутбук для работы и учебы	img/thinkpad.jpg
	3	Футболка Adidas	1999.50	Спортивная футболка для пов...	img/adidas_tshirt.jpg
	4	Книга "Война и мир"	599.00	Роман Л. Н. Толстого	img/war_and_peace.jp
	5	Конструктор LEGO	4999.00	Конструктор для детей от 6 лет	img/lego.jpg
	6	Кофейная кружка	399.00	Керамическая кружка 350мл	img/mug.jpg
*	NULL	NULL	NULL	NULL	NULL

1. Символ * означает «все столбцы».
2. `SELECT * FROM products;`
3. Что происходит:
 - выбираются все поля таблицы
 - возвращаются все строки
4. В production не рекомендуется использовать *, лучше указывать конкретные поля.

ПСЕВДОНИМЫ (AS)

ПЕРЕИМЕНОВАНИЕ СТОЛБЦОВ В РЕЗУЛЬТАТЕ ЗАПРОСА



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
1 SELECT
2     product_name AS "Название",
3     product_price AS "Цена"
4 FROM storedb.products;
```

The result grid displays the output of the query, with columns named "Название" and "Цена". The first two columns are highlighted with red boxes. The data rows are as follows:

Название	Цена
Смартфон Samsung Galaxy S21	79999.99
Ноутбук Lenovo ThinkPad	120000.00
Футболка Adidas	1999.50
Книга "Война и мир"	599.00
Конструктор LEGO	4999.00
Кофейная кружка	399.00

1. AS задаёт временное имя столбца в выводе.

а. SELECT

б. product_name **AS** "Название",

в. product_price **AS** "Цена"

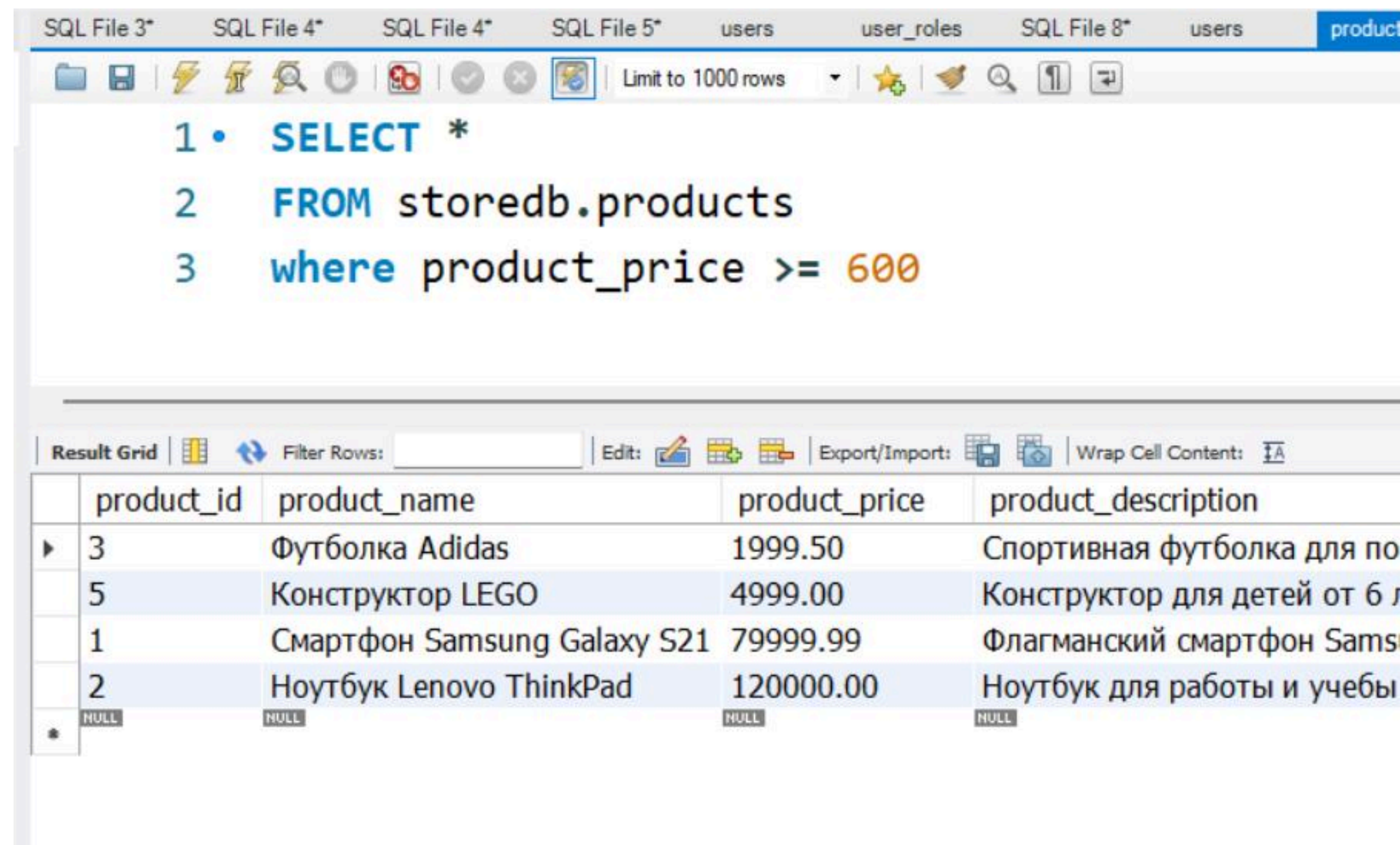
г. FROM products;

2. Можно без AS:

а. SELECT product_name "Название" FROM products;

ФИЛЬТРАЦИЯ ДАННЫХ — WHERE

ВЫБОР СТРОК ПО УСЛОВИЮ



The screenshot shows a database management tool interface. At the top, there are several tabs labeled 'SQL File 3*', 'SQL File 4*', 'SQL File 4*', 'SQL File 5*', 'users', 'user_roles', 'SQL File 8*', 'users', and 'product'. Below the tabs is a toolbar with various icons. The main area displays an SQL query:

```
1 • SELECT *
2 FROM storedb.products
3 where product_price >= 600
```

Below the query is a 'Result Grid' section. It includes a 'Filter Rows:' input field, an 'Edit:' button, an 'Export/Import:' button, and a 'Wrap Cell Content:' checkbox. The grid itself contains the following data:

	product_id	product_name	product_price	product_description
▶	3	Футболка Adidas	1999.50	Спортивная футболка для по
	5	Конструктор LEGO	4999.00	Конструктор для детей от 6 л
	1	Смартфон Samsung Galaxy S21	79999.99	Флагманский смартфон Sams
	2	Ноутбук Lenovo ThinkPad	120000.00	Ноутбук для работы и учебы
*	NULL	NULL	NULL	NULL

1. Оператор WHERE

a. SELECT *

b. FROM products

c. **WHERE** product_price > 5000;

2. Операторы сравнения:

- = - равенство
- <, > - меньше/больше
- <=, >= - меньше/больше либо равно
- != - неравенство

ПОИСК ПО ДИАПАЗОНУ И СПИСКУ

BETWEEN И IN

1. Оператор **BETWEEN** используется для поиска значений в диапазоне.
2. BETWEEN работает с числами, датами и строками
 - a. **WHERE** поле **BETWEEN** значение1 **AND** значение2
 - b. **WHERE** поле $\geq$ значение1 **AND** поле $\leq$ значение2

```
1 • SELECT product_name, product_price
2 FROM storedb.products
3 WHERE product_price BETWEEN 200 AND 5000;
```

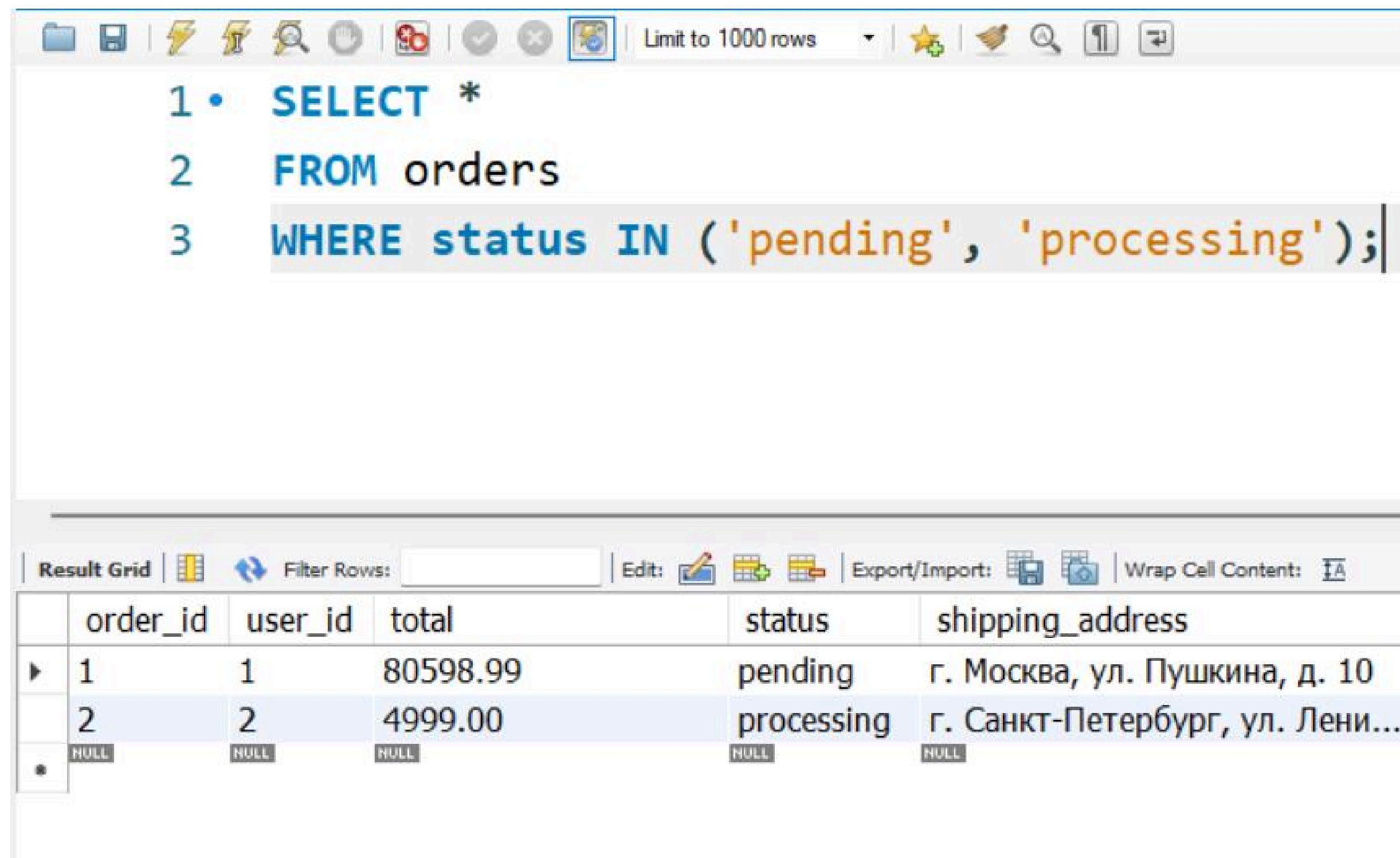
product_name	product_price
Кофейная кружка	399.00
Книга "Война и мир"	599.00
Футболка Adidas	1999.50
Конструктор LEGO	4999.00

```
1 • SELECT *
2 FROM orders
3 WHERE created_at BETWEEN '2026-01-01' AND '2026-02-21';
```

order_id	user_id	total	status	shipping_address	created_at	updated_at
1	1	80598.99	pending	г. Москва, ул. Пушкина, д. 10	2026-02-07 14:02:29	2026-02-07 14:02:29
2	2	4999.00	processing	г. Санкт-Петербург, ул. Лени...	2026-02-07 14:02:29	2026-02-07 14:02:29
•	NULL	NULL	NULL	NULL	NULL	NULL

ПОИСК ПО ДИАПАЗОНУ И СПИСКУ

BETWEEN И IN



The screenshot shows a database query editor with a toolbar at the top. The SQL query is as follows:

```
1 • SELECT *
2 FROM orders
3 WHERE status IN ('pending', 'processing');
```

Below the query editor is a "Result Grid" showing the results of the query. The grid has columns for order_id, user_id, total, status, and shipping_address. The first two rows are highlighted in blue.

	order_id	user_id	total	status	shipping_address
▶	1	1	80598.99	pending	г. Москва, ул. Пушкина, д. 10
	2	2	4999.00	processing	г. Санкт-Петербург, ул. Лени...
*	NULL	NULL	NULL	NULL	NULL

1. Оператор **IN** — для поиска значений в конкретном списке.

a. **WHERE** поле **IN** (значение1, значение2, значение3)

2. Пример

a. **WHERE** status **IN** ('pending', 'processing');

b. **WHERE** status = 'pending' **OR** status = 'processing'

3. Когда использовать?

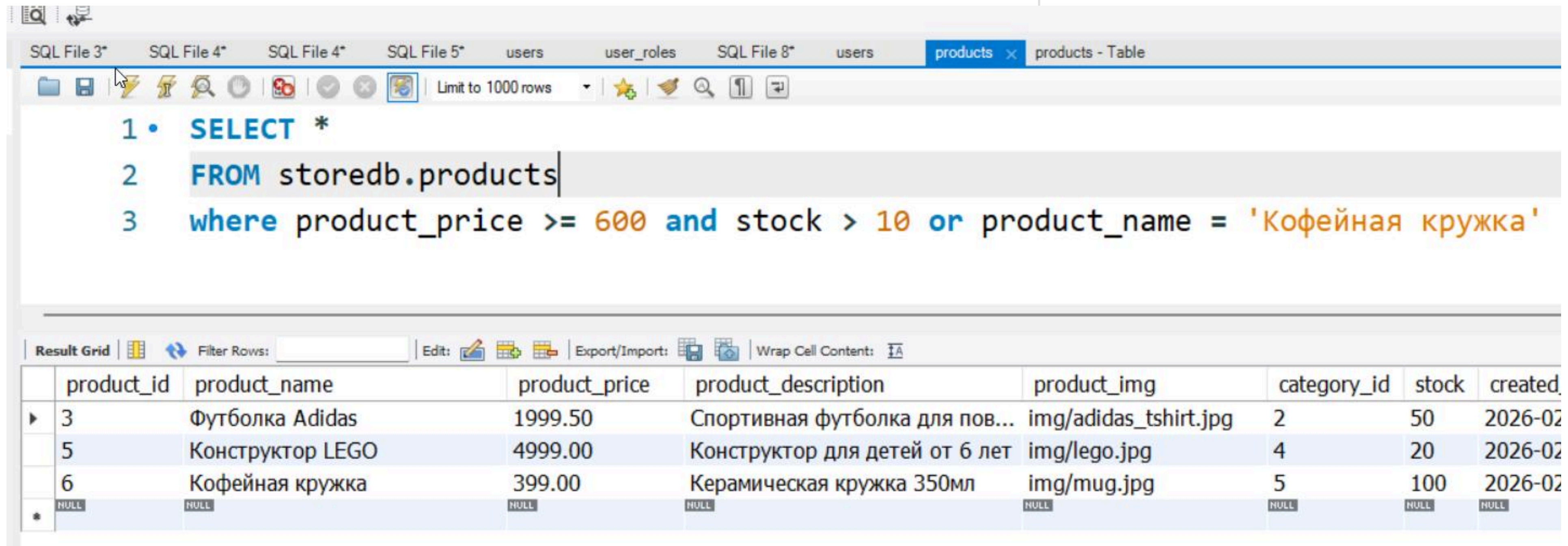
- BETWEEN — диапазоны (цена, дата, возраст)
- IN — фиксированный список значений (статусы, роли, категории)

ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

ИСПОЛЬЗОВАНИЕ AND, OR, NOT

1. Операторы:

- AND – оба условия
- OR – одно из условий
- NOT – отрицание



The screenshot shows a database management tool interface. The top toolbar includes icons for file operations and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
1 • SELECT *
2 FROM storedb.products
3 where product_price >= 600 and stock > 10 or product_name = 'Кофейная кружка'
```

Below the editor is the 'Result Grid' section, which displays the query results in a table format. The table has columns for product_id, product_name, product_price, product_description, product_img, category_id, stock, and created. The results show three rows of data, with the third row highlighted in blue.

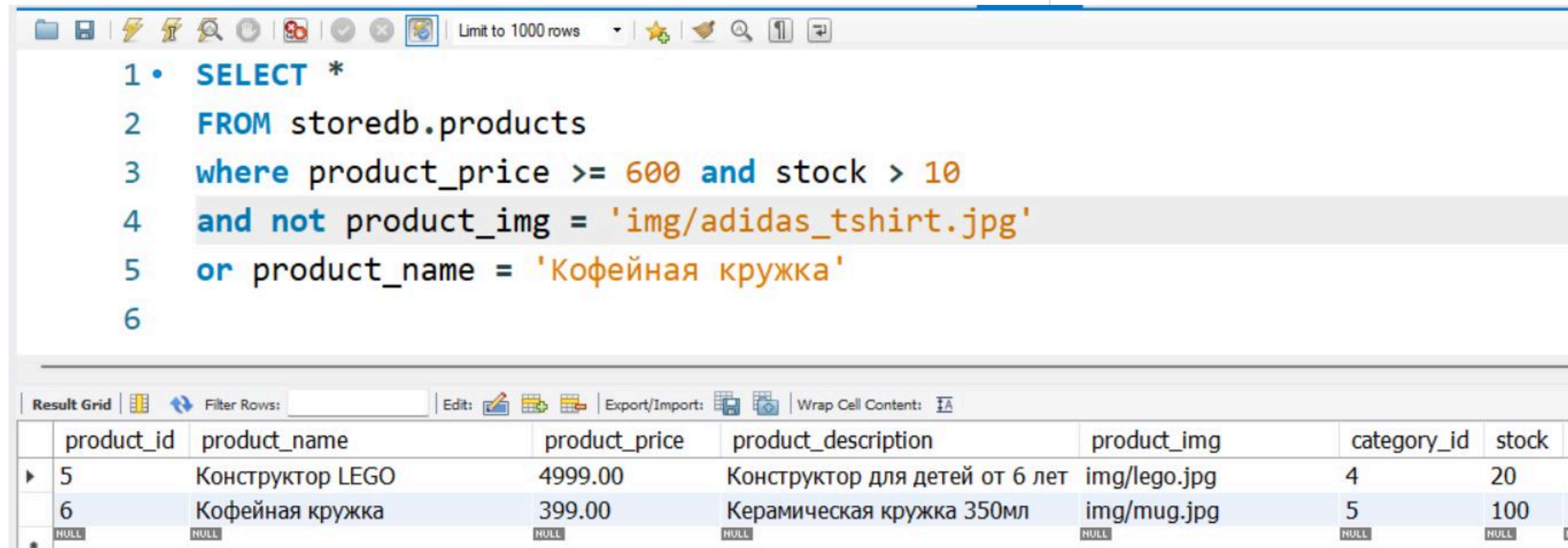
	product_id	product_name	product_price	product_description	product_img	category_id	stock	created
▶	3	Футболка Adidas	1999.50	Спортивная футболка для пов...	img/adidas_tshirt.jpg	2	50	2026-02
	5	Конструктор LEGO	4999.00	Конструктор для детей от 6 лет	img/lego.jpg	4	20	2026-02
	6	Кофейная кружка	399.00	Керамическая кружка 350мл	img/mug.jpg	5	100	2026-02
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ОПЕРАТОР NOT В SELECT

ИСПОЛЬЗОВАНИЕ AND, OR, NOT

1. Операторы:

- AND – оба условия
- OR – одно из условий
- NOT – отрицание



The screenshot shows a database query editor with a toolbar at the top containing icons for file operations, search, and execution. The query is as follows:

```
1 • SELECT *
2 FROM storedb.products
3 where product_price >= 600 and stock > 10
4 and not product_img = 'img/adidas_tshirt.jpg'
5 or product_name = 'Кофейная кружка'
6
```

Below the query, the 'Result Grid' is displayed with a toolbar for filtering, editing, and exporting. The results table has the following data:

	product_id	product_name	product_price	product_description	product_img	category_id	stock	
▶	5	Конструктор LEGO	4999.00	Конструктор для детей от 6 лет	img/lego.jpg	4	20	2
	6	Кофейная кружка	399.00	Керамическая кружка 350мл	img/mug.jpg	5	100	2
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NI

ПОИСК ПО ШАБЛОНУ — LIKE

ЧАСТИЧНОЕ СОВПАДЕНИЕ СТРОК

1. Оператор **LIKE** используется для поиска по шаблону.

2. Специальные символы:

- % — любое количество символов
- _ — один любой символ

```
1 • SELECT user_name, user_email
2   FROM users
3   WHERE user_email LIKE '%@mail.ru';
```

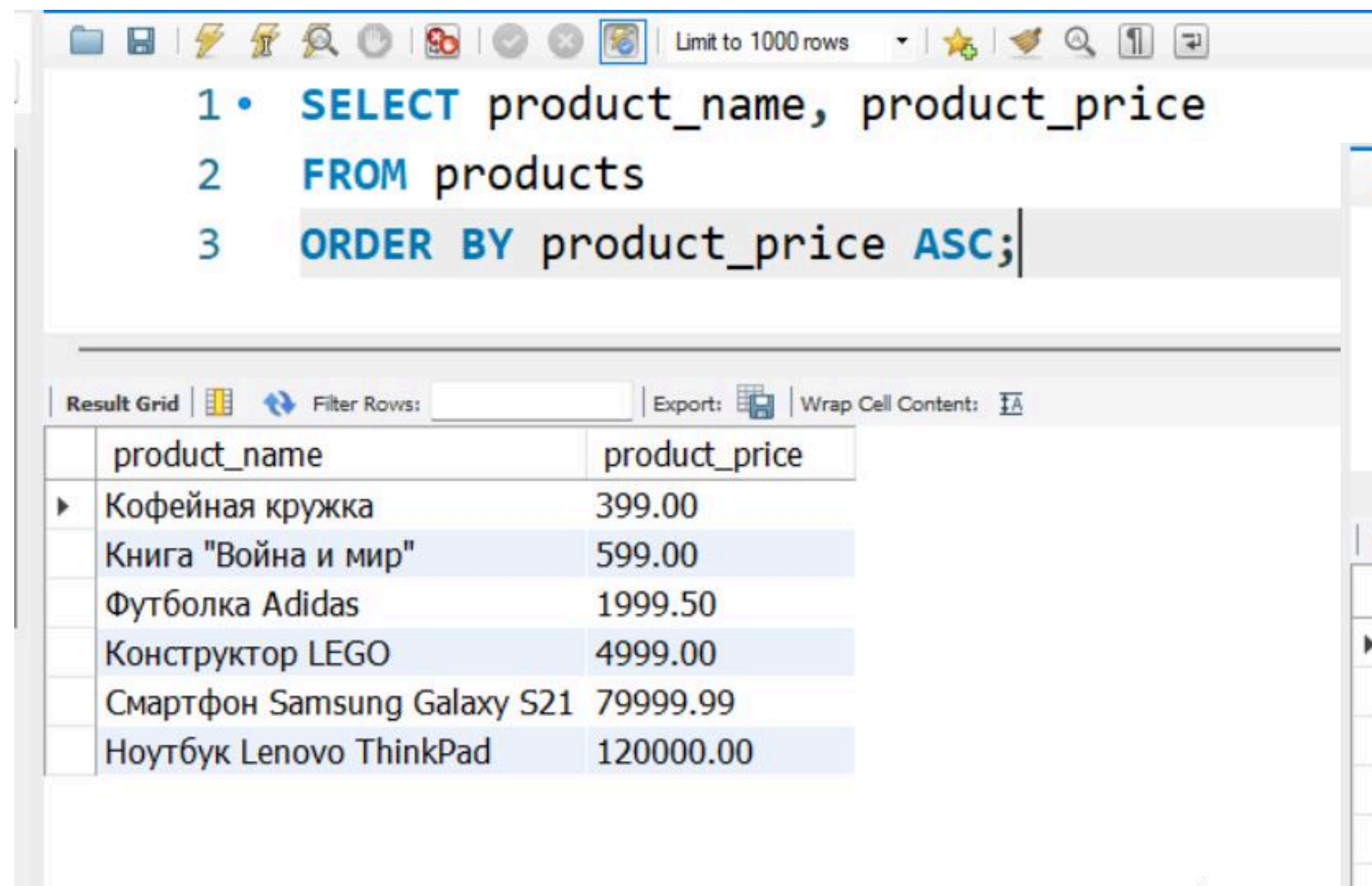
	user_name	user_email
▶	Иван Иванов	ivan@mail.ru
	Мария Петрова	maria@mail.ru
	Администратор	admin@mail.ru

```
1 • SELECT *
2   FROM products
3   WHERE product_name LIKE 'Смартфон%';
```

	product_id	product_name	product_price	product_description
▶	1	Смартфон Samsung Galaxy S21	79999.99	Флагманский смартфон Samsu... i
*	NULL	NULL	NULL	NULL

СОРТИРОВКА — ORDER BY

УПРАВЛЕНИЕ ПОРЯДКОМ ВЫВОДА ДАННЫХ



1 • **SELECT** product_name, product_price
2 **FROM** products
3 **ORDER BY** product_price **ASC**;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	product_name	product_price
▶	Кофейная кружка	399.00
	Книга "Война и мир"	599.00
	Футболка Adidas	1999.50
	Конструктор LEGO	4999.00
	Смартфон Samsung Galaxy S21	79999.99
	Ноутбук Lenovo ThinkPad	120000.00

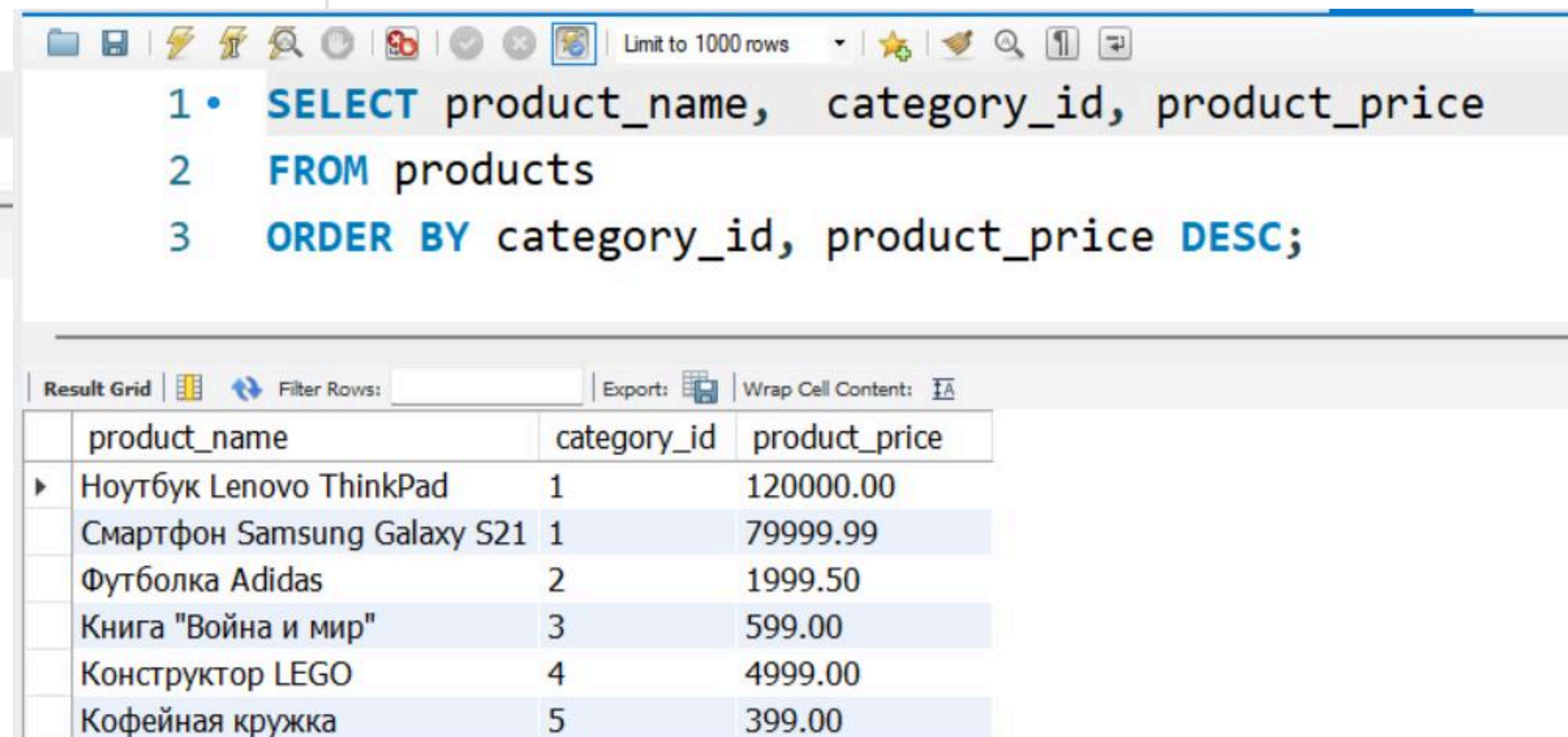
1. По умолчанию MySQL **не гарантирует** порядок строк.

2. Синтаксис:

a. ORDER BY поле **ASC|DESC**

b. ASC — по возрастанию (по умолчанию)

c. DESC — по убыванию



1 • **SELECT** product_name, category_id, product_price
2 **FROM** products
3 **ORDER BY** category_id, product_price **DESC**;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	product_name	category_id	product_price
▶	Ноутбук Lenovo ThinkPad	1	120000.00
	Смартфон Samsung Galaxy S21	1	79999.99
	Футболка Adidas	2	1999.50
	Книга "Война и мир"	3	599.00
	Конструктор LEGO	4	4999.00
	Кофейная кружка	5	399.00

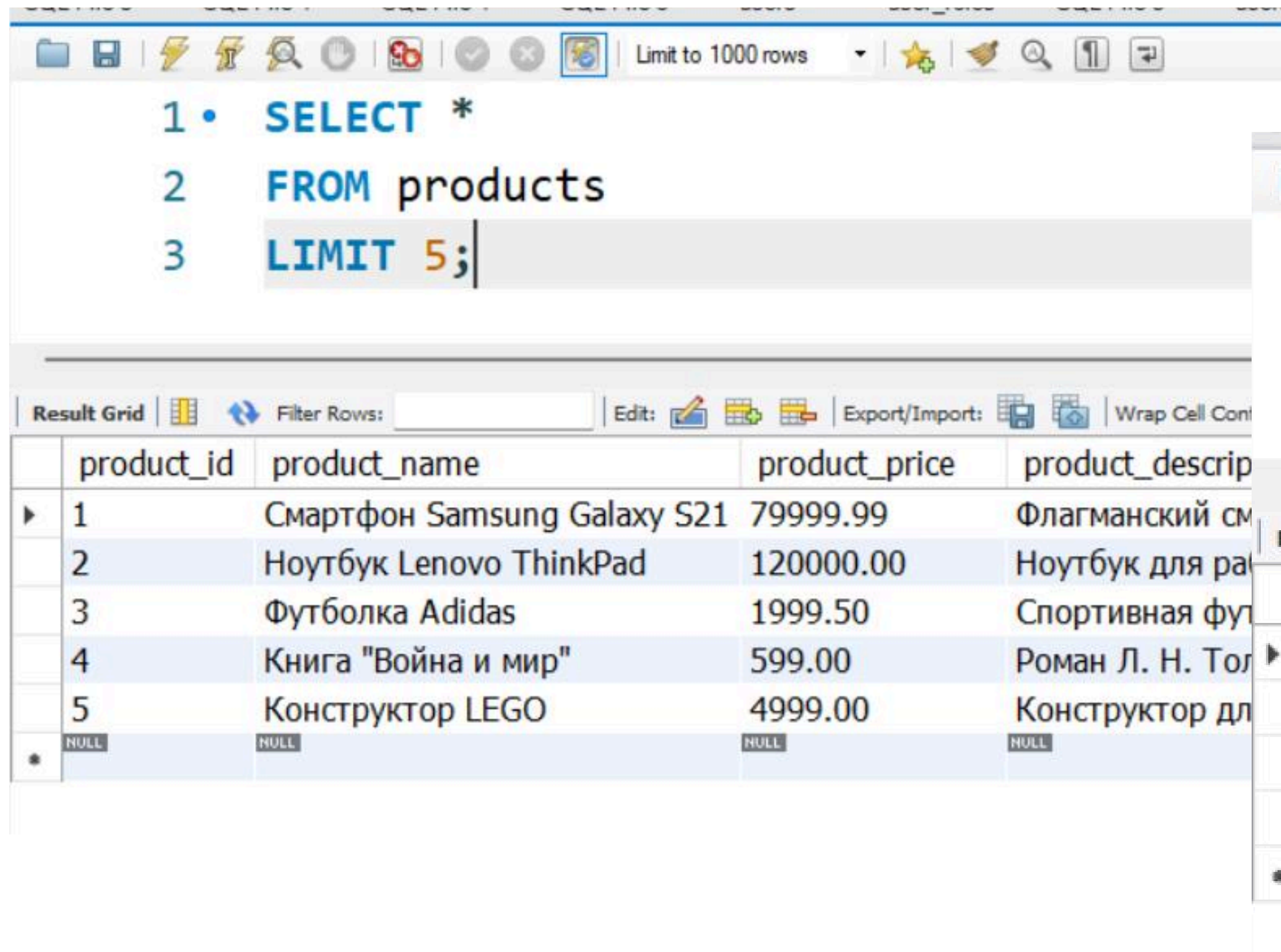
Сначала сортировка по категории, внутри — по цене.

ОГРАНИЧЕНИЕ КОЛИЧЕСТВА СТРОК — LIMIT

ПОЛУЧЕНИЕ ЧАСТИ РЕЗУЛЬТАТОВ

1. Используется для:

- пагинации
- вывода ТОП-N
- тестирования

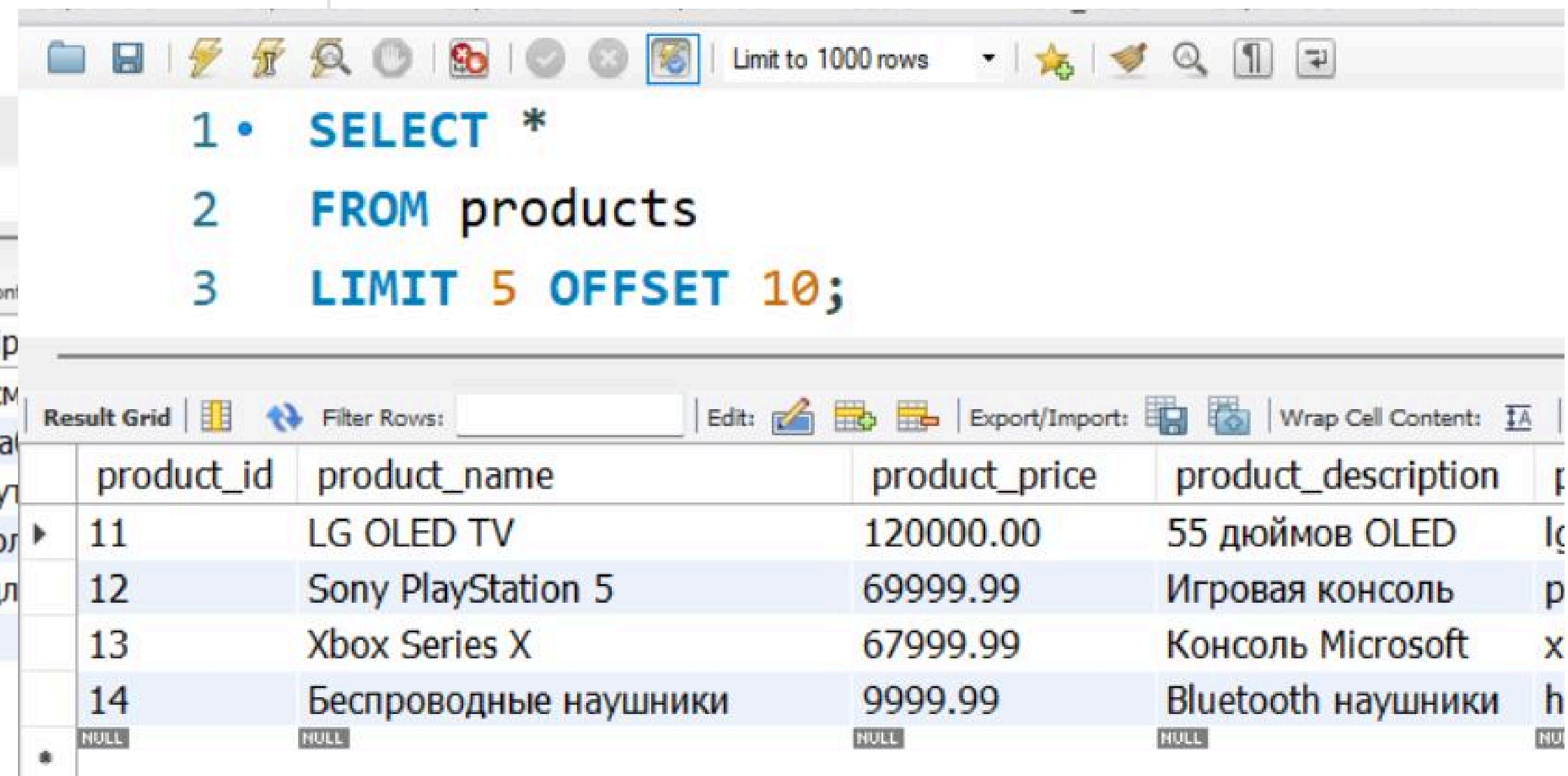


SQL Query:

```
1 • SELECT *
2 FROM products
3 LIMIT 5;
```

Result Grid:

	product_id	product_name	product_price	product_description
▶	1	Смартфон Samsung Galaxy S21	79999.99	Флагманский см
	2	Ноутбук Lenovo ThinkPad	120000.00	Ноутбук для ра
	3	Футболка Adidas	1999.50	Спортивная фу
	4	Книга "Война и мир"	599.00	Роман Л. Н. Тол
	5	Конструктор LEGO	4999.00	Конструктор дл
*	NULL	NULL	NULL	NULL



SQL Query:

```
1 • SELECT *
2 FROM products
3 LIMIT 5 OFFSET 10;
```

Result Grid:

	product_id	product_name	product_price	product_description
▶	11	LG OLED TV	120000.00	55 дюймов OLED
	12	Sony PlayStation 5	69999.99	Игровая консоль
	13	Xbox Series X	67999.99	Консоль Microsoft
	14	Беспроводные наушники	9999.99	Bluetooth наушники
*	NULL	NULL	NULL	NULL

АГРЕГАТНЫЕ ФУНКЦИИ

ВЫЧИСЛЕНИЕ ИТОГОВЫХ ЗНАЧЕНИЙ

The image displays four screenshots of a SQL IDE, each showing a query and its result. The IDE interface includes a toolbar with icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' section.

- Top Left:** Query: `1 • SELECT COUNT(*) AS total_products`
`2 FROM products;`
Result Grid:

total_products
14
- Top Right:** Query: `1 • SELECT AVG(total) AS avg_order_total`
`2 FROM orders;`
Result Grid:

avg_order_total
75449.618750
- Bottom Left:** Query: `1 • SELECT SUM(total) AS delivered_revenue`
`2 FROM orders`
`3 WHERE status = 'delivered';`
Result Grid:

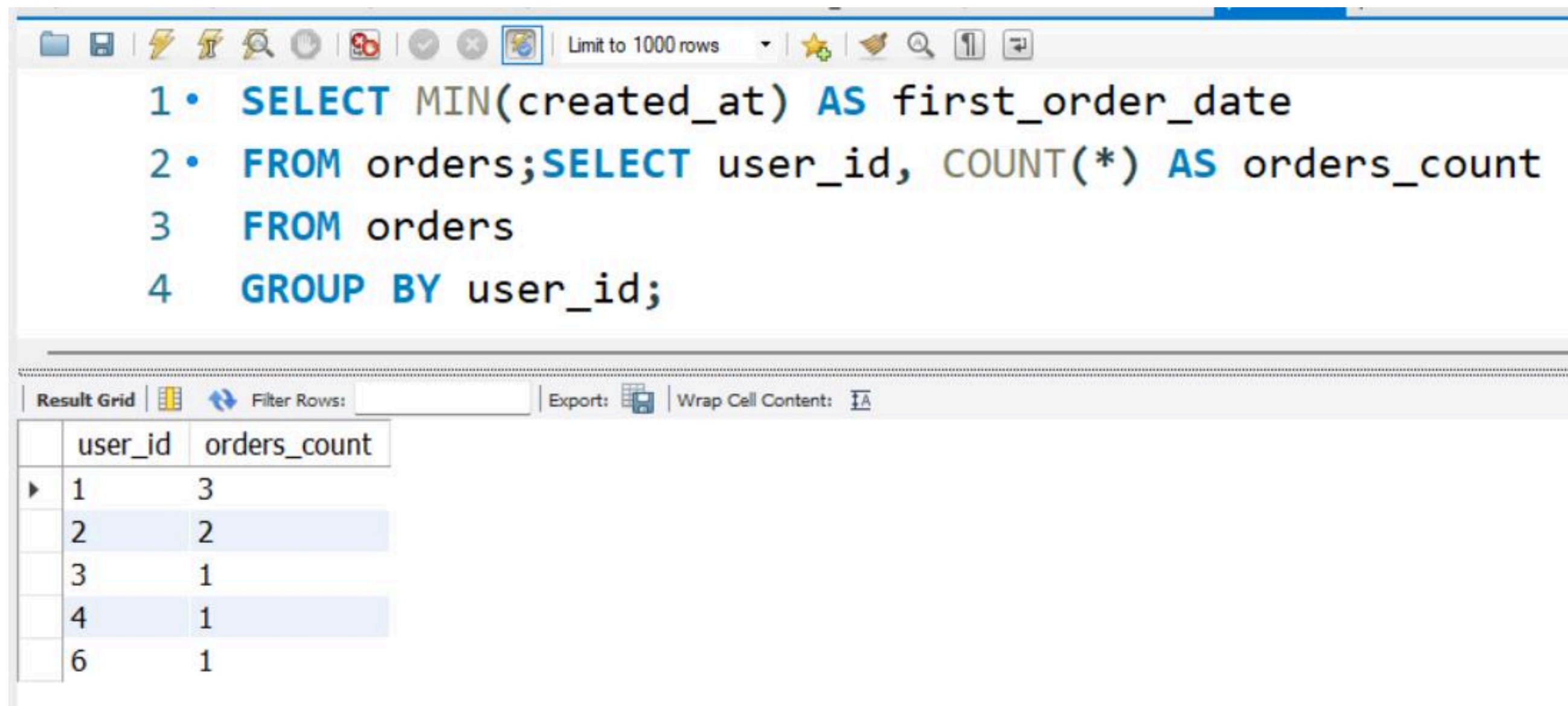
delivered_revenue
219999.99
- Bottom Right:** Query: `1 • SELECT MIN(created_at) AS first_order_date`
`2 FROM orders;`
Result Grid:

first_order_date
2026-02-07 14:02:29

1. Агрегатные функции в SQL используются для вычисления итоговых значений по набору строк.
2. Они работают не с одной записью, а сразу с группой строк (вся таблица или результат WHERE / GROUP BY).
3. Чаще всего применяются для аналитики, отчётов, статистики и бизнес-логики.
4. Основные функции:
 - а. COUNT() – Подсчёт количества строк
 - б. SUM() – Сумма значений
 - в. AVG() – Среднее значение
 - г. MIN() – Минимальное значение
 - д. MAX() – Максимальное значение

ГРУППИРОВКА — GROUP BY

ГРУППИРОВКА ДАННЫХ И АГРЕГИРОВАНИЕ



```
1 • SELECT MIN(created_at) AS first_order_date
2 • FROM orders; SELECT user_id, COUNT(*) AS orders_count
3 FROM orders
4 GROUP BY user_id;
```

	user_id	orders_count
▶	1	3
	2	2
	3	1
	4	1
	6	1

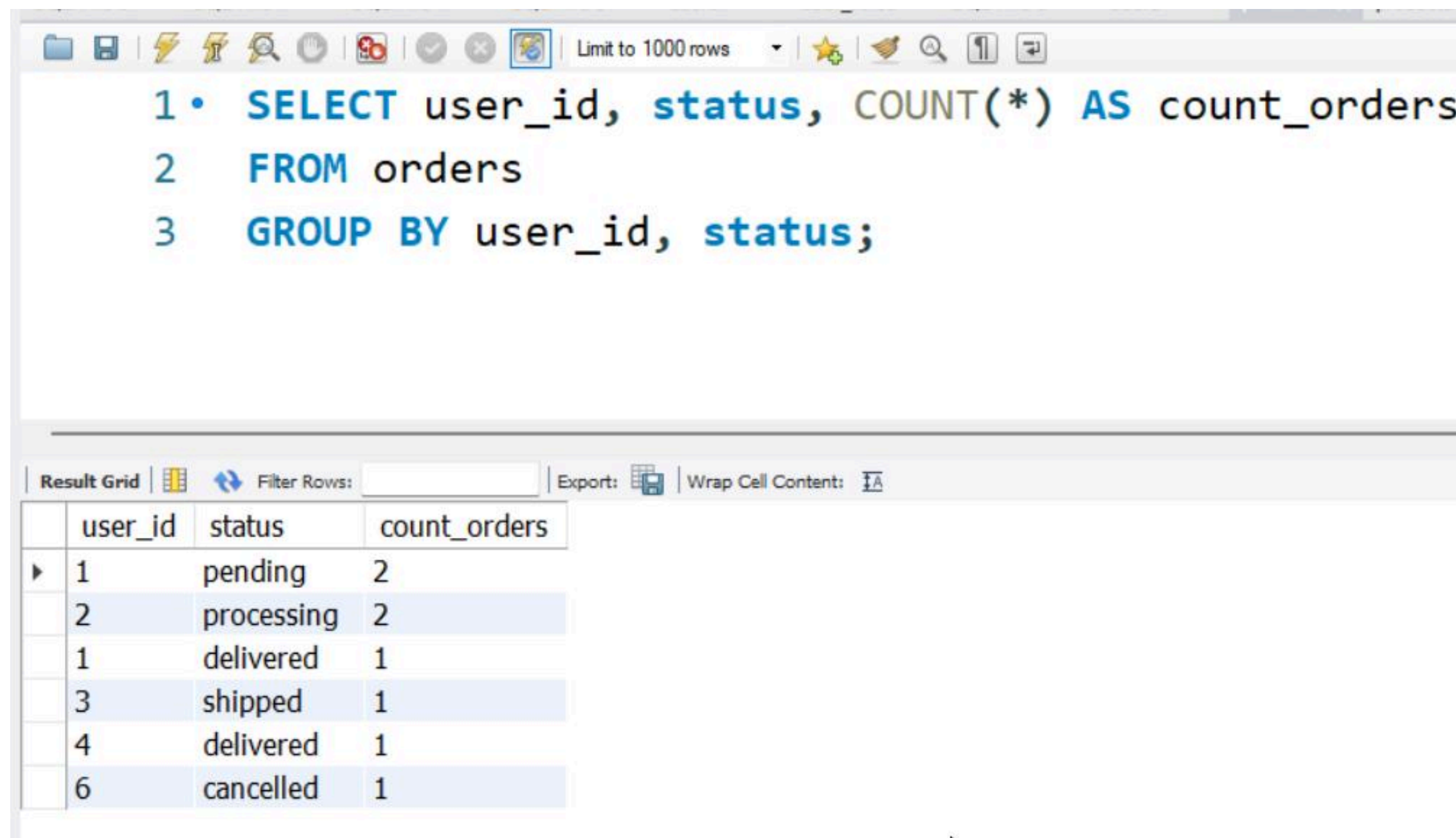
Пример: Сколько заказов у каждого пользователя

1. Все заказы группируются по `user_id`
2. Для каждой группы считается количество строк

1. **GROUP BY** используется для объединения строк с одинаковыми значениями в группы и последующего применения агрегатных функций к каждой группе.
2. Без **GROUP BY** агрегатная функция работает по всей таблице.
3. С **GROUP BY** — по каждой группе отдельно.
4. Базовый синтаксис
 - a. **SELECT** column_name, **AGGREGATE_FUNCTION**(column)
 - b. **FROM** table_name
 - c. **GROUP BY** column_name;
5. Логика выполнения запроса:
 - a. **FROM** — выбирается таблица
 - b. **WHERE** — фильтрация строк
 - c. **GROUP BY** — группировка
 - d. **SELECT** — вычисление агрегатов
 - e. **ORDER BY** — сортировка

НЕСКОЛЬКО ПОЛЕЙ В GROUP BY

МОЖНО ГРУППИРОВАТЬ ПО НЕСКОЛЬКИМ КОЛОНКАМ



The screenshot shows a SQL query editor with the following query:

```
1 • SELECT user_id, status, COUNT(*) AS count_orders
2 FROM orders
3 GROUP BY user_id, status;
```

Below the query, the results are displayed in a table with the following columns: user_id, status, and count_orders.

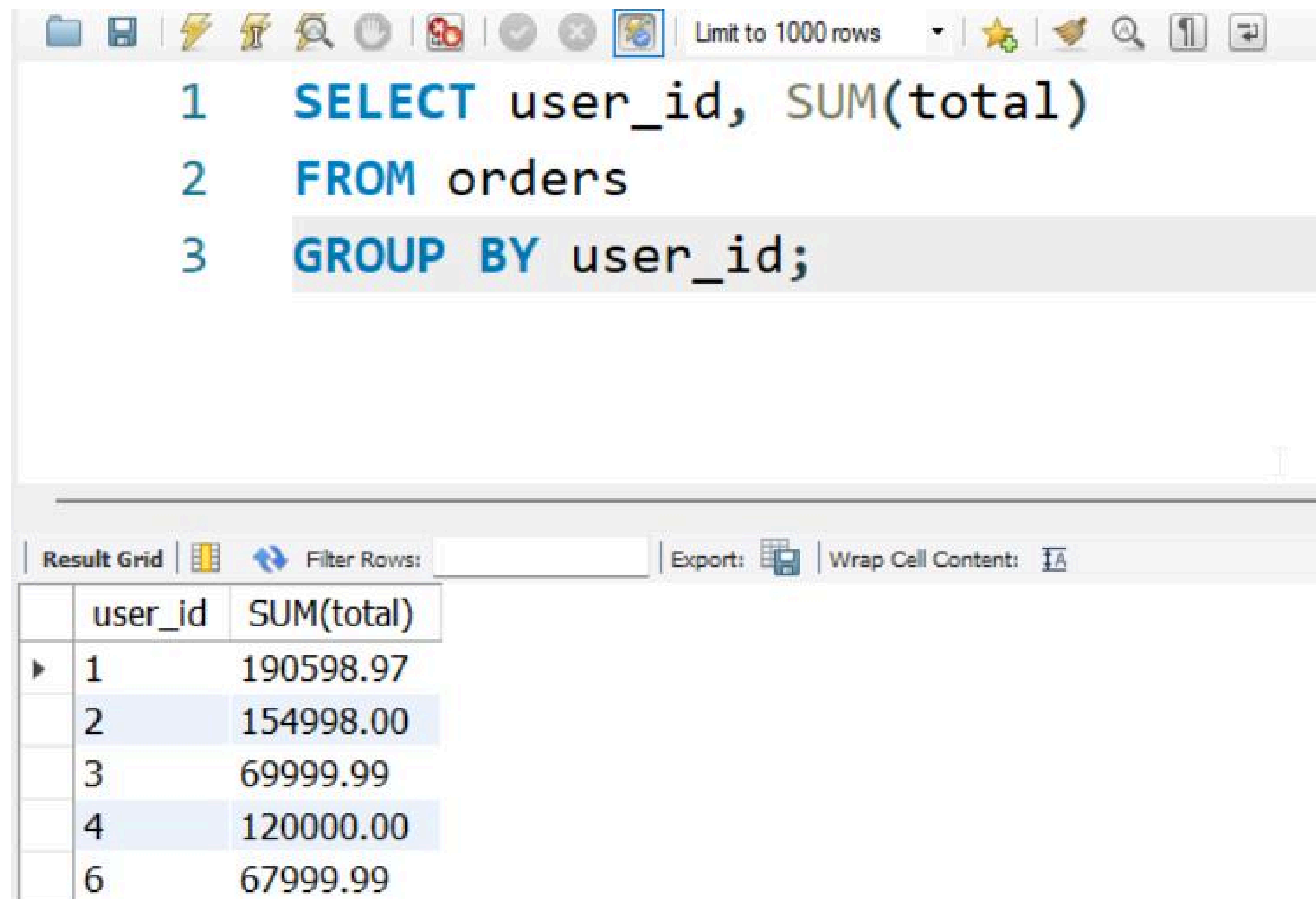
	user_id	status	count_orders
▶	1	pending	2
	2	processing	2
	1	delivered	1
	3	shipped	1
	4	delivered	1
	6	cancelled	1

Количество заказов по пользователю и статусу

1. **GROUP BY** используется для объединения строк с одинаковыми значениями в группы и последующего применения агрегатных функций к каждой группе.
2. Без **GROUP BY** агрегатная функция работает по всей таблице.
3. С **GROUP BY** — по каждой группе отдельно.
4. Базовый синтаксис
 - a. **SELECT** column_name, **AGGREGATE_FUNCTION**(column)
 - b. **FROM** table_name
 - c. **GROUP BY** column_name;
5. Логика выполнения запроса:
 - a. **FROM** — выбирается таблица
 - b. **WHERE** — фильтрация строк
 - c. **GROUP BY** — группировка
 - d. **SELECT** — вычисление агрегатов
 - e. **ORDER BY** — сортировка

ОШИБКА НАЧИНАЮЩИХ

ГРУППИРОВКА ДАННЫХ И АГРЕГИРОВАНИЕ



The screenshot shows a SQL IDE window with a toolbar at the top. The query editor contains the following SQL code:

```
1 SELECT user_id, SUM(total)
2 FROM orders
3 GROUP BY user_id;
```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are displayed in a table with two columns: 'user_id' and 'SUM(total)'. The table contains five rows of data.

	user_id	SUM(total)
▶	1	190598.97
	2	154998.00
	3	69999.99
	4	120000.00
	6	67999.99

1. Нельзя выбирать столбцы, которых нет в GROUP BY и которые не являются агрегатами.

2. Неправильно:

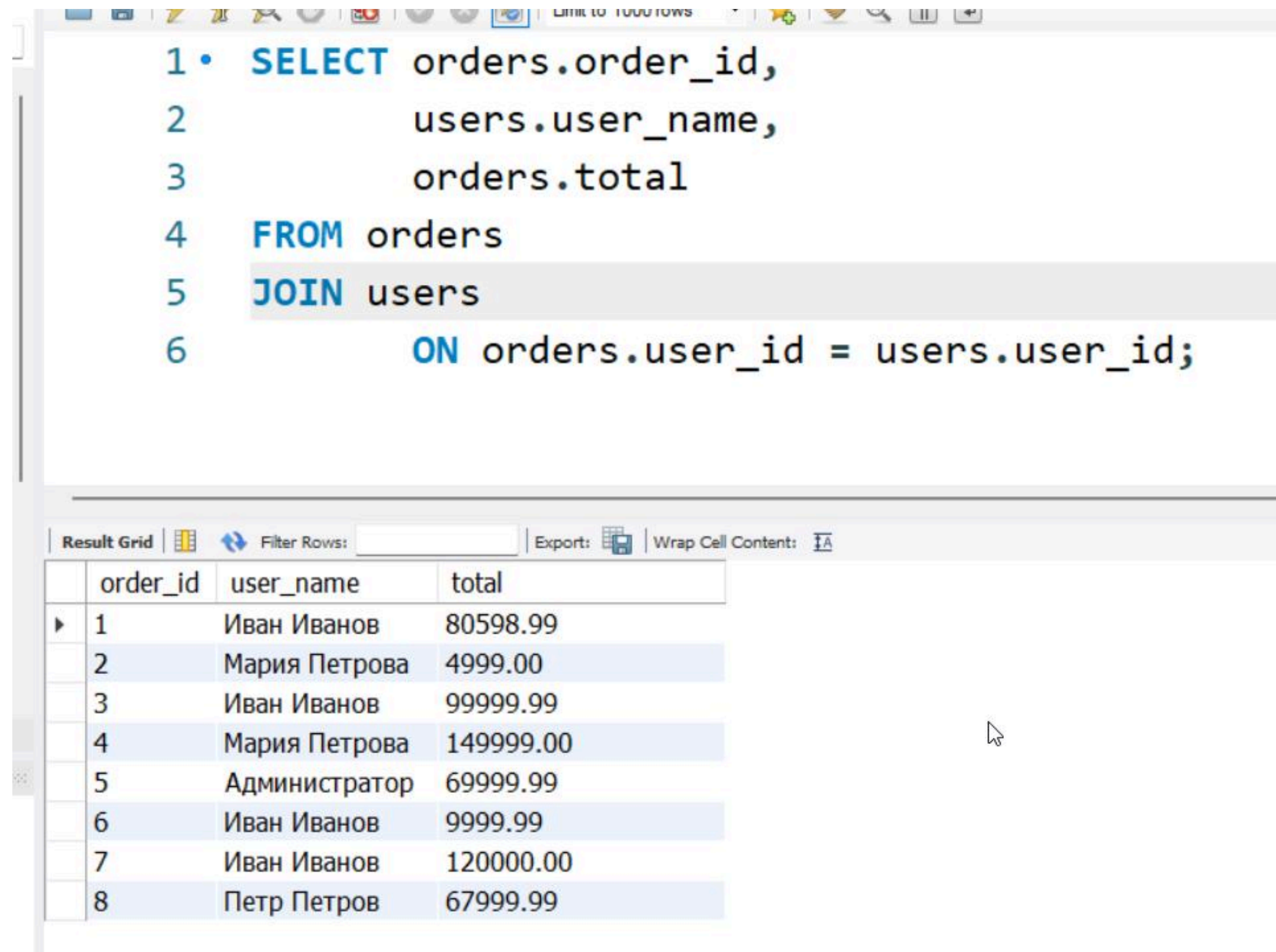
- a. SELECT user_id, total
- b. FROM orders
- c. GROUP BY user_id;
- d. Почему ошибка?
 - total не агрегирован
 - total не входит в GROUP BY

3. Правильно:

- a. SELECT user_id, SUM(total)
- b. FROM orders
- c. GROUP BY user_id;

ОБЪЕДИНЕНИЕ ТАБЛИЦ – JOIN

ПОЛУЧЕНИЕ ДАННЫХ ИЗ НЕСКОЛЬКИХ ТАБЛИЦ



The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
1 • SELECT orders.order_id,  
2       users.user_name,  
3       orders.total  
4 FROM orders  
5 JOIN users  
6     ON orders.user_id = users.user_id;
```

Below the query, the results are displayed in a table with the following columns: order_id, user_name, and total. The results are as follows:

order_id	user_name	total
1	Иван Иванов	80598.99
2	Мария Петрова	4999.00
3	Иван Иванов	99999.99
4	Мария Петрова	149999.00
5	Администратор	69999.99
6	Иван Иванов	9999.99
7	Иван Иванов	120000.00
8	Петр Петров	67999.99

1. Зачем нужен **JOIN**

2. Пример задачи:

а. Показать список **заказов** с именем **пользователя**.

б. Информация хранится в двух таблицах:

- orders.user_id
- users.user_id

3. Связь осуществляется через внешний ключ.

4. Основной синтаксис **JOIN**

а. **SELECT** columns

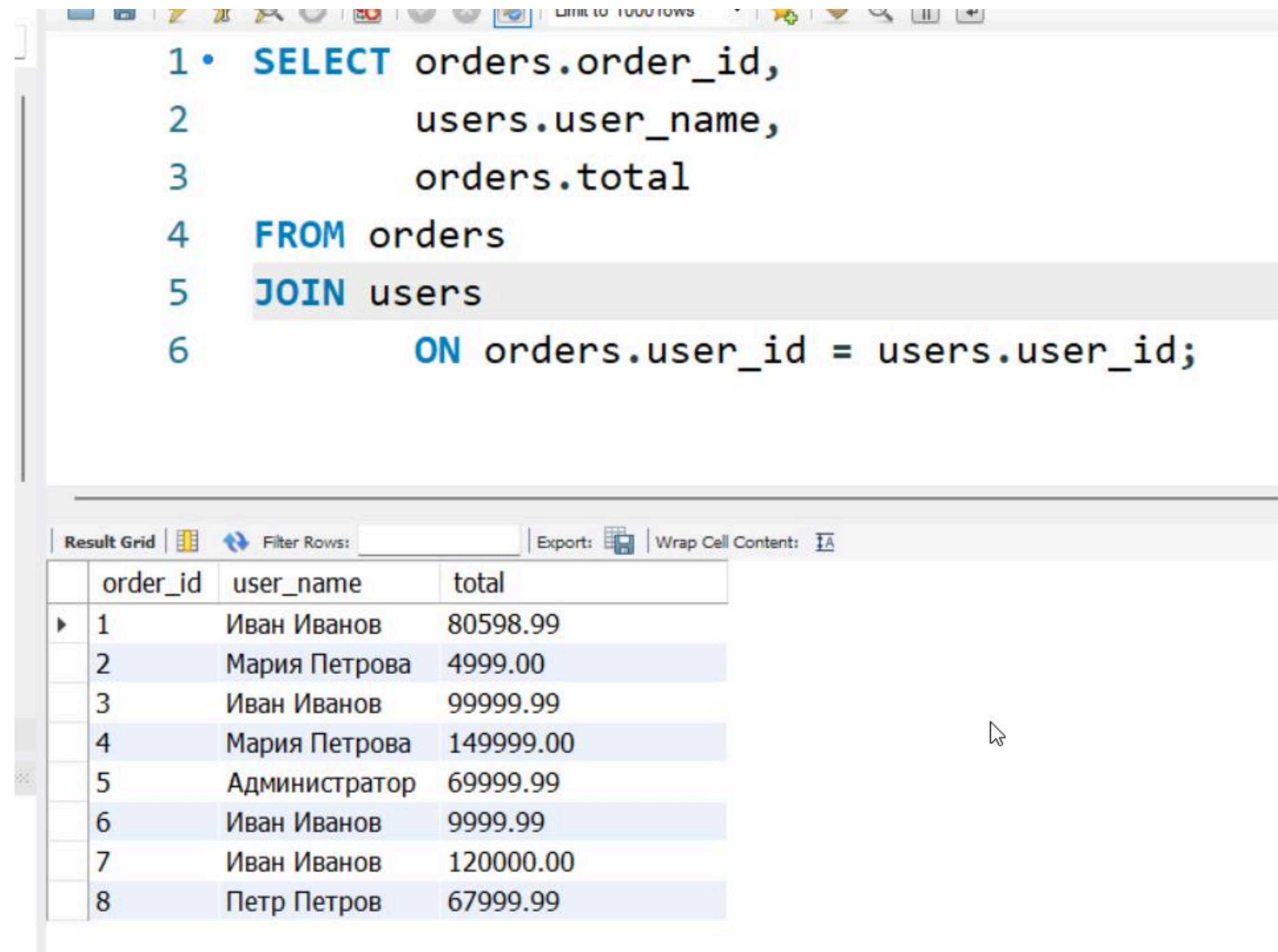
б. **FROM** table1

с. **JOIN** table2 **ON** table1.column = table2.column;

5. **ON** – условие соединения таблиц.

INNER JOIN

ТОЛЬКО СОВПАДАЮЩИЕ ЗАПИСИ



```
1 • SELECT orders.order_id,  
2       users.user_name,  
3       orders.total  
4 FROM orders  
5 JOIN users  
6     ON orders.user_id = users.user_id;
```

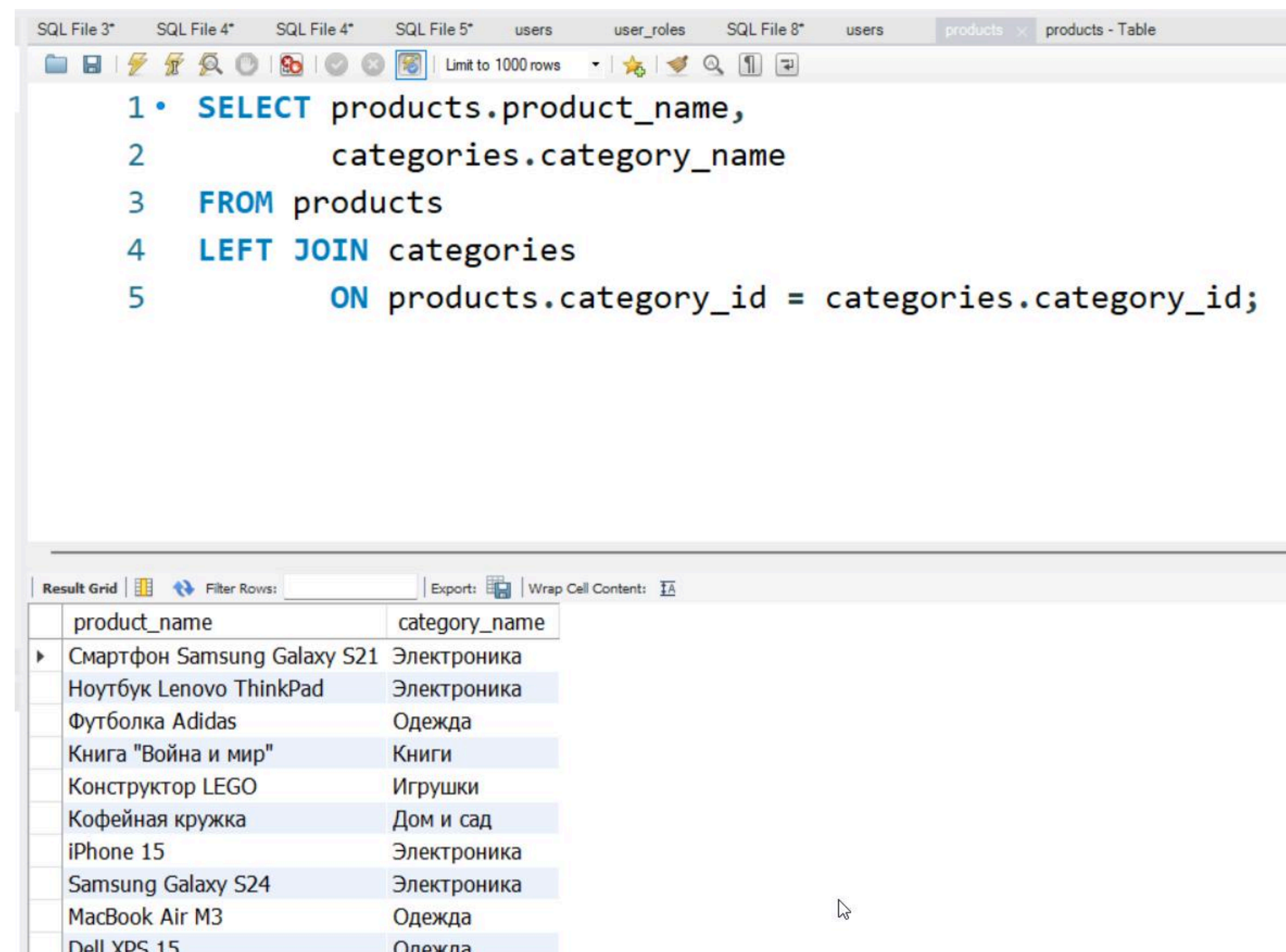
Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	order_id	user_name	total
▶	1	Иван Иванов	80598.99
	2	Мария Петрова	4999.00
	3	Иван Иванов	99999.99
	4	Мария Петрова	149999.00
	5	Администратор	69999.99
	6	Иван Иванов	9999.99
	7	Иван Иванов	120000.00
	8	Петр Петров	67999.99

1. Возвращает строки, которые есть в обеих таблицах.
2. Пример: Заказы с именами пользователей
3. Что произойдёт:
 - Будут показаны только те заказы, у которых существует пользователь.
4. Если пользователь удалён – заказ не попадёт в результат.

LEFT JOIN

ВСЕ ДАННЫЕ ИЗ ЛЕВОЙ ТАБЛИЦЫ



The screenshot shows a SQL IDE window with a query editor and a results grid. The query is a LEFT JOIN between the 'products' and 'categories' tables. The results grid displays the following data:

product_name	category_name
Смартфон Samsung Galaxy S21	Электроника
Ноутбук Lenovo ThinkPad	Электроника
Футболка Adidas	Одежда
Книга "Война и мир"	Книги
Конструктор LEGO	Игрушки
Кофейная кружка	Дом и сад
iPhone 15	Электроника
Samsung Galaxy S24	Электроника
MacBook Air M3	Одежда
Dell XPS 15	Одежда

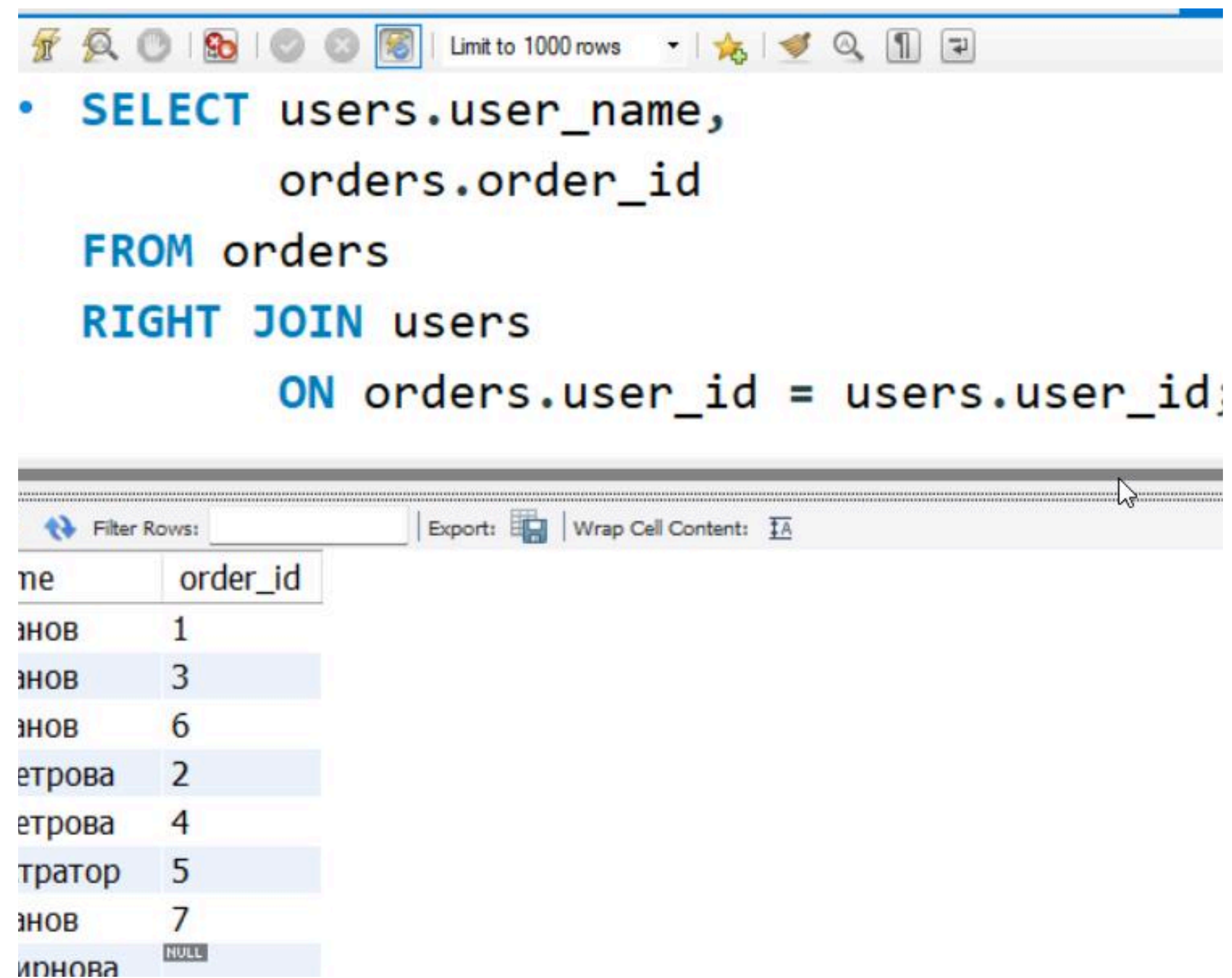
1. Возвращает:

- все строки из левой таблицы
- совпадения из правой
- если совпадения нет → NULL

2. Пример: Все товары и их категории

RIGHT JOIN

ВСЕ ДАННЫЕ ИЗ ПРАВОЙ ТАБЛИЦЫ



The screenshot shows a database query editor with a toolbar at the top. The query is as follows:

```
• SELECT users.user_name,  
      orders.order_id  
FROM orders  
RIGHT JOIN users  
ON orders.user_id = users.user_id;
```

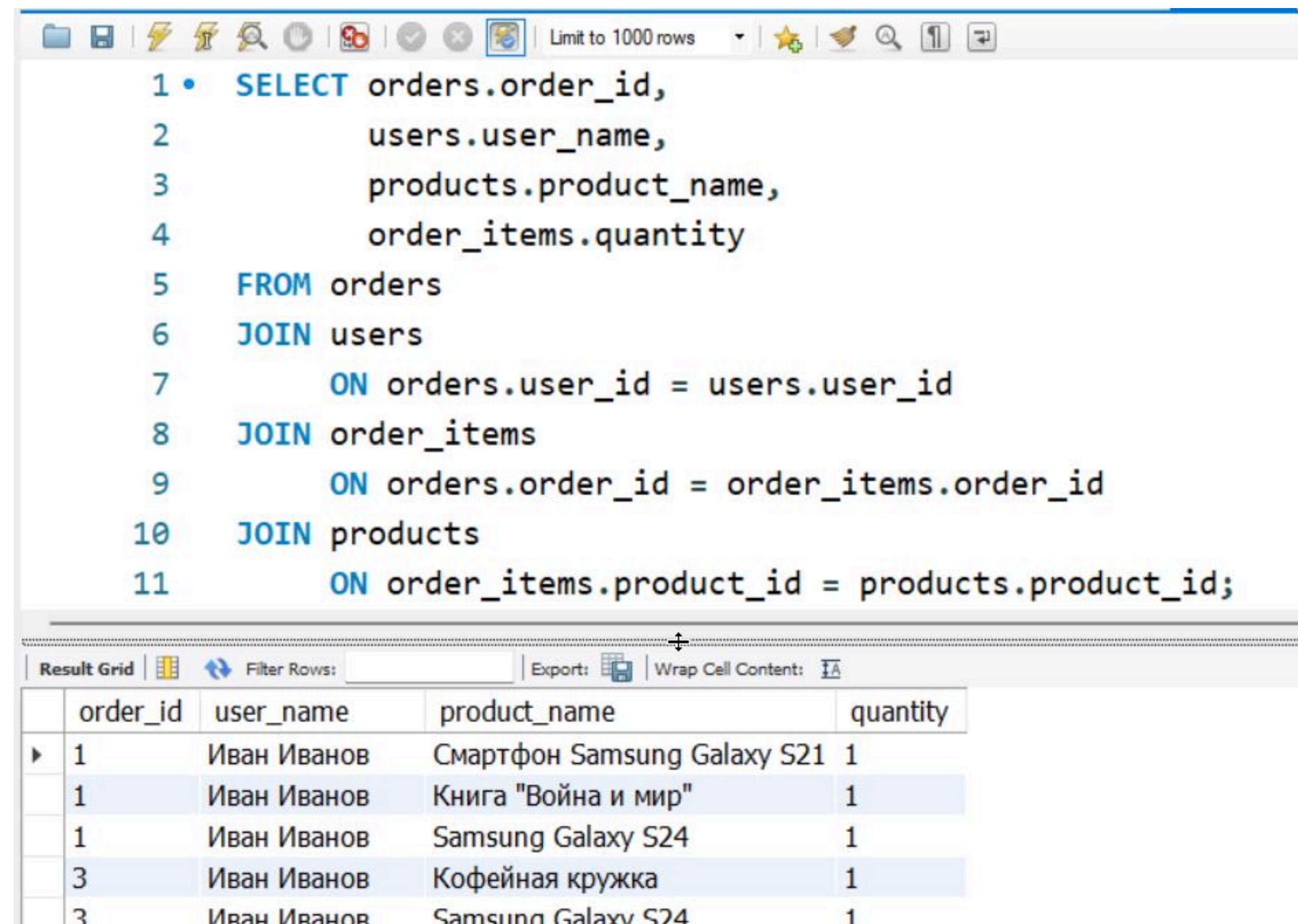
Below the query, the results are displayed in a table. The table has two columns: 'user_name' and 'order_id'. The results are as follows:

user_name	order_id
Янов	1
Янов	3
Янов	6
Петрова	2
Петрова	4
Тратор	5
Янов	7
Ипнова	NULL

1. Редко используется, но логика зеркальная LEFT JOIN.
2. Покажет всех пользователей, даже если у них нет заказов.

JOIN НЕСКОЛЬКИХ ТАБЛИЦ

ЦЕПОЧКА СОЕДИНЕНИЙ



The screenshot shows a database query editor with a SQL query and its results. The query is a chain of joins connecting four tables: orders, users, order_items, and products. The results table shows five rows of data, each representing an order item with its order ID, user name, product name, and quantity.

```
1 • SELECT orders.order_id,  
2         users.user_name,  
3         products.product_name,  
4         order_items.quantity  
5 FROM orders  
6 JOIN users  
7     ON orders.user_id = users.user_id  
8 JOIN order_items  
9     ON orders.order_id = order_items.order_id  
10 JOIN products  
11     ON order_items.product_id = products.product_id;
```

	order_id	user_name	product_name	quantity
▶	1	Иван Иванов	Смартфон Samsung Galaxy S21	1
	1	Иван Иванов	Книга "Война и мир"	1
	1	Иван Иванов	Samsung Galaxy S24	1
	3	Иван Иванов	Кофейная кружка	1
	3	Иван Иванов	Samsung Galaxy S24	1

1. Можно объединять более двух таблиц.

2. Пример: Детали заказа

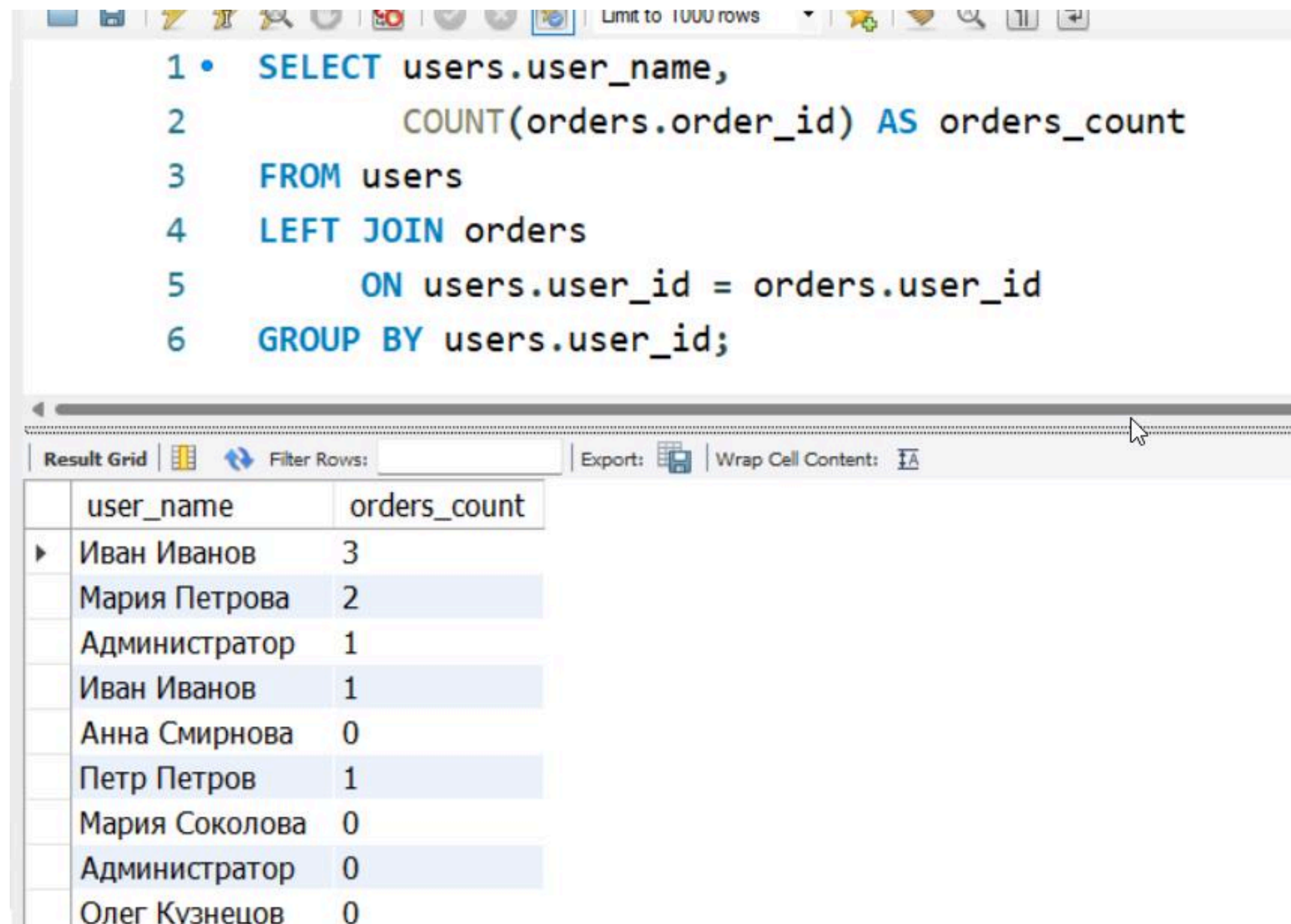
а. Показать:

- номер заказа
- имя пользователя
- название товара
- количество

3. Это уже полноценный отчёт.

JOIN + GROUP BY

ЦЕПОЧКА СОЕДИНЕНИЙ



The screenshot shows a database query editor with a SQL query and its results. The query is as follows:

```
1 • SELECT users.user_name,  
2      COUNT(orders.order_id) AS orders_count  
3 FROM users  
4 LEFT JOIN orders  
5     ON users.user_id = orders.user_id  
6 GROUP BY users.user_id;
```

The results are displayed in a table with two columns: user_name and orders_count. The table contains 10 rows of data, including some duplicates for user names.

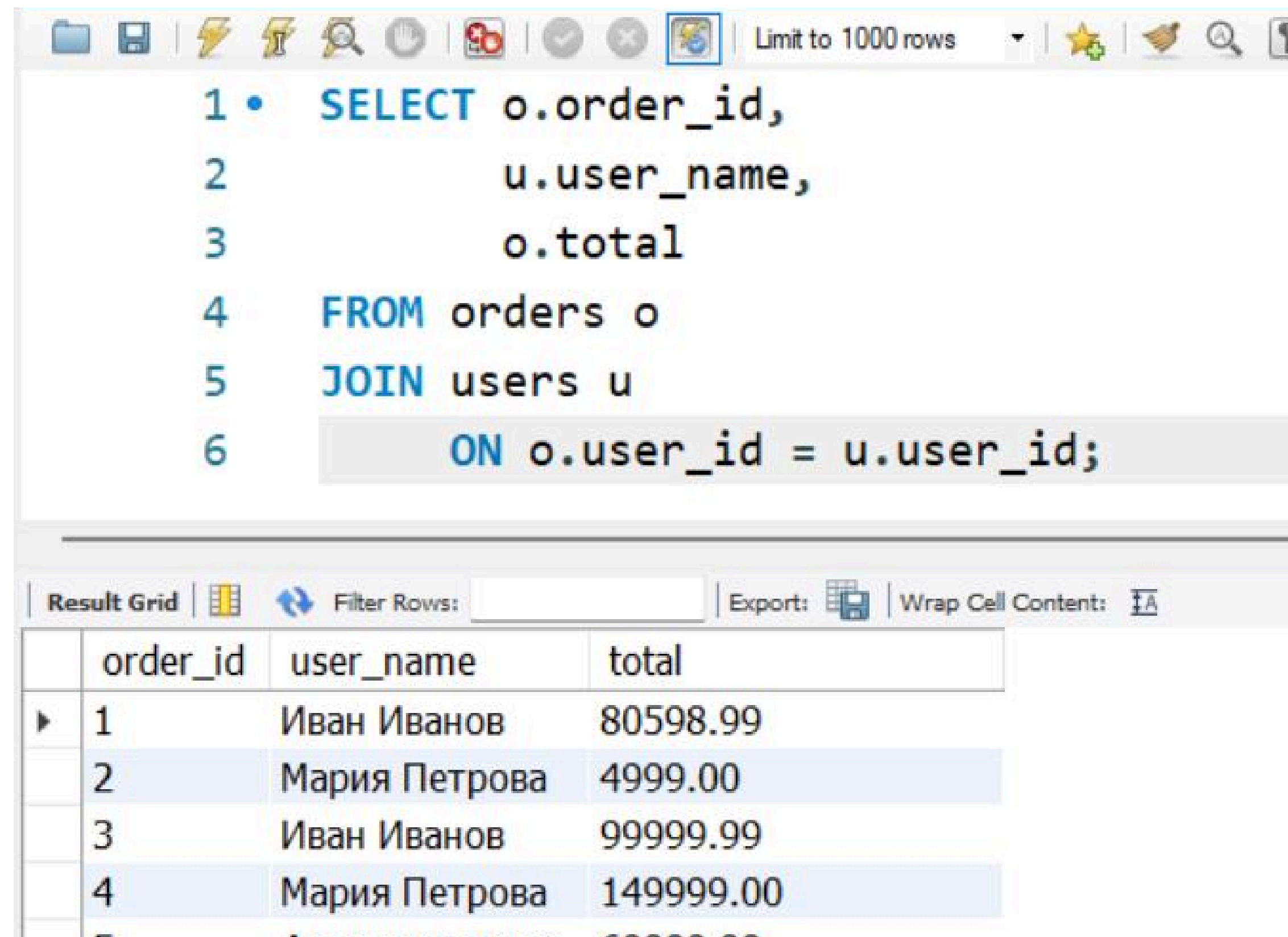
user_name	orders_count
Иван Иванов	3
Мария Петрова	2
Администратор	1
Иван Иванов	1
Анна Смирнова	0
Петр Петров	1
Мария Соколова	0
Администратор	0
Олег Кузнецов	0

1. Пример: Сколько заказов у каждого пользователя
2. Здесь важно:
 - LEFT JOIN позволяет показать пользователей без заказов
 - COUNT() считает количество заказов

1. Чтобы не писать длинные названия таблиц.
2. Это профессиональный стиль написания SQL.

JOIN + АЛИАСЫ (AS)

УПРОЩЕНИЕ ЧИТАЕМОСТИ ЗАПРОСА



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
1 • SELECT o.order_id,  
2         u.user_name,  
3         o.total  
4 FROM orders o  
5 JOIN users u  
6     ON o.user_id = u.user_id;
```

Below the editor is the 'Result Grid' section, which includes a 'Filter Rows' input and an 'Export' button. The grid displays the results of the query:

	order_id	user_name	total
▶	1	Иван Иванов	80598.99
	2	Мария Петрова	4999.00
	3	Иван Иванов	99999.99
	4	Мария Петрова	149999.00

SELECT В РЕАЛЬНОМ ПРОЕКТЕ

КОМПЛЕКСНЫЙ ЗАПРОС

The screenshot shows a database management interface with a left sidebar displaying the schema structure. The main window contains a SQL editor with a complex query and a result grid below it.

Schema Structure (Left Sidebar):

- justice_db
 - storedb
 - Tables
 - categories
 - order_items
 - orders
 - products
 - Columns
 - product_id
 - product_name
 - product_price
 - product_description
 - product_img
 - category_id
 - stock
 - created_at
 - updated_at
 - Indexes
 - Foreign Keys
 - Triggers
 - shopping_cart
 - users
 - Columns
 - user_id
 - user_name

SQL Query (Editor):

```
1 • SELECT
2     p.product_name,
3     p.product_price,
4     c.category_name
5 FROM products p
6 JOIN categories c
7     ON p.category_id = c.category_id
8 WHERE p.product_price BETWEEN 300 and 100000
9 ORDER BY p.product_price DESC
10 LIMIT 10;
```

Result Grid:

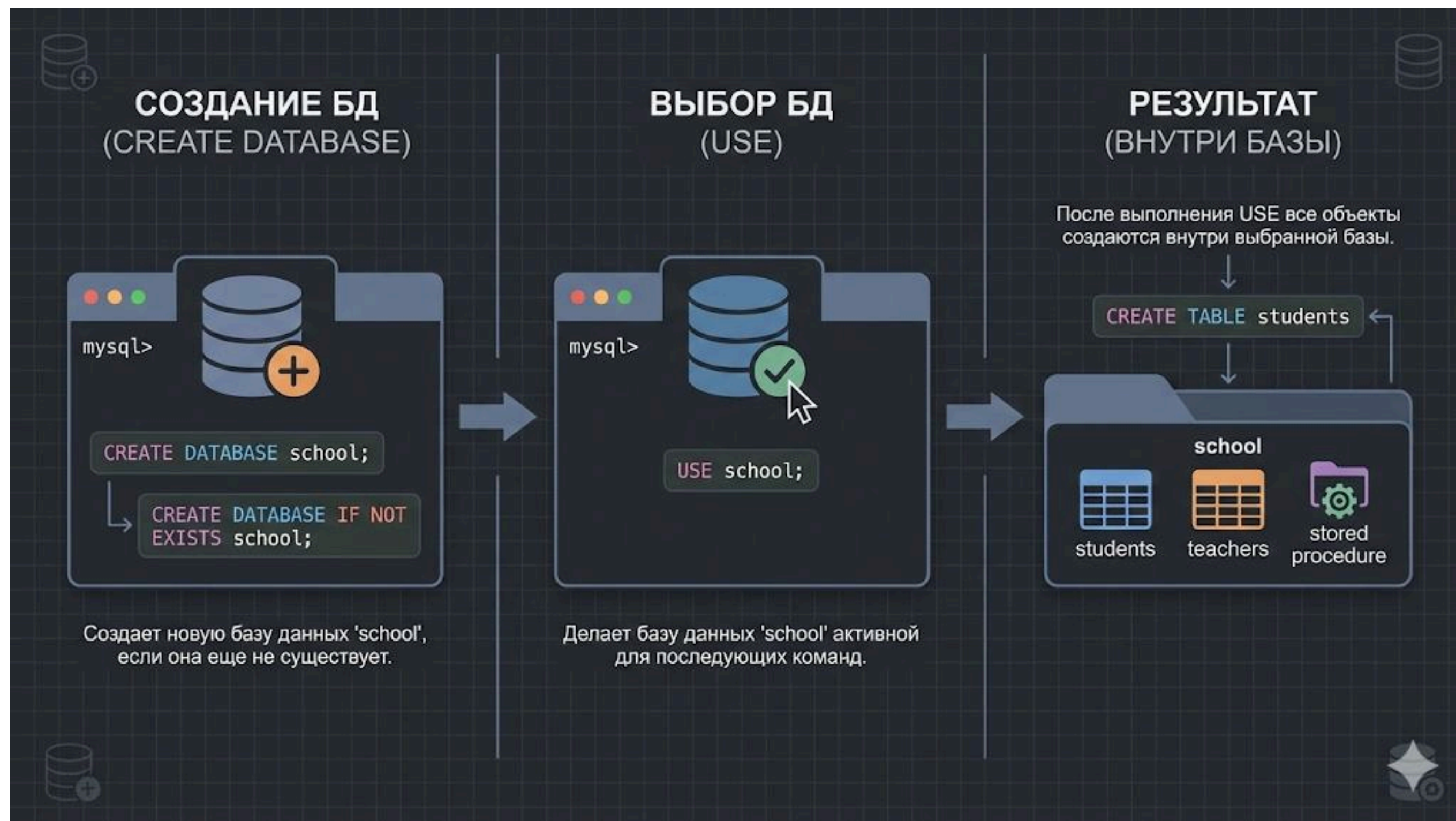
product_name	product_price	category_name
iPhone 15	99999.99	Электроника
Samsung Galaxy S24	89999.99	Электроника
Смартфон Samsung Galaxy S21	79999.99	Электроника
Sony PlayStation 5	69999.99	Дом и сад
Xbox Series X	67999.99	Дом и сад
Беспроводные наушники	9999.99	Игрушки
Конструктор LEGO	4999.00	Игрушки
Футболка Adidas	1999.50	Одежда
Книга "Война и мир"	599.00	Книги
Кофейная кружка	399.00	Дом и сад

1. Что делает:

- соединяет товары и категории
- сортирует по цене
- ограничение по цене от 300 до 100000
- выводит 10 самых дорогих

КОМАНДА CREATE — НАЗНАЧЕНИЕ

ОСНОВНЫЕ ЭЛЕМЕНТЫ СИНТАКСИСА SQL



1. Команда CREATE используется для создания объектов базы данных:

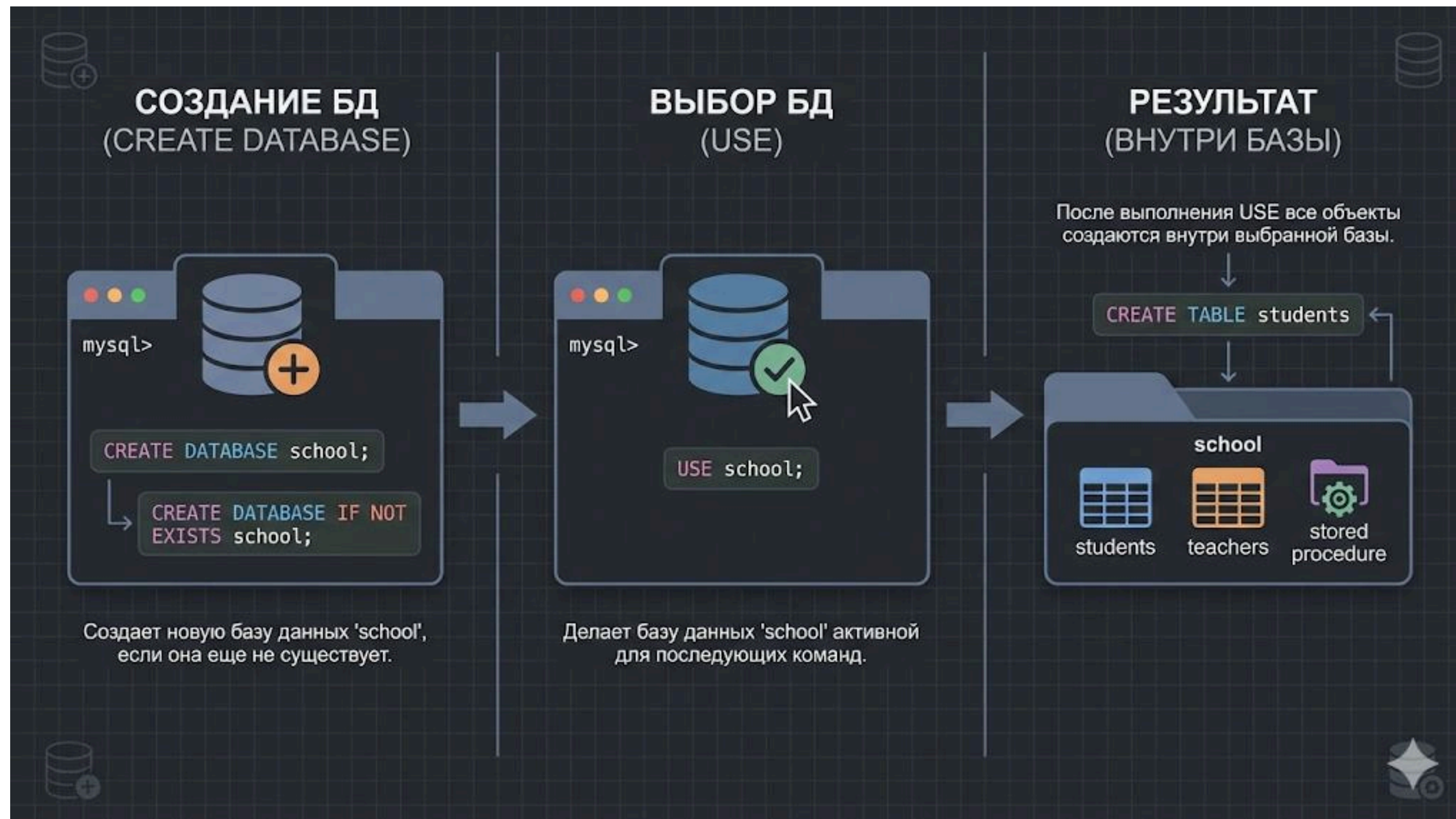
- CREATE DATABASE — создать базу данных
- CREATE TABLE — создать таблицу
- CREATE INDEX — создать индекс
- CREATE VIEW — создать представление

2. Синтаксис общий:

- CREATE объект имя_объекта (...);

CREATE DATABASE

СОЗДАНИЕ И ВЫБОР БАЗЫ ДАННЫХ ДЛЯ РАБОТЫ



1. Создание БД:

а. `CREATE DATABASE school;`

2. Если база может уже существовать:

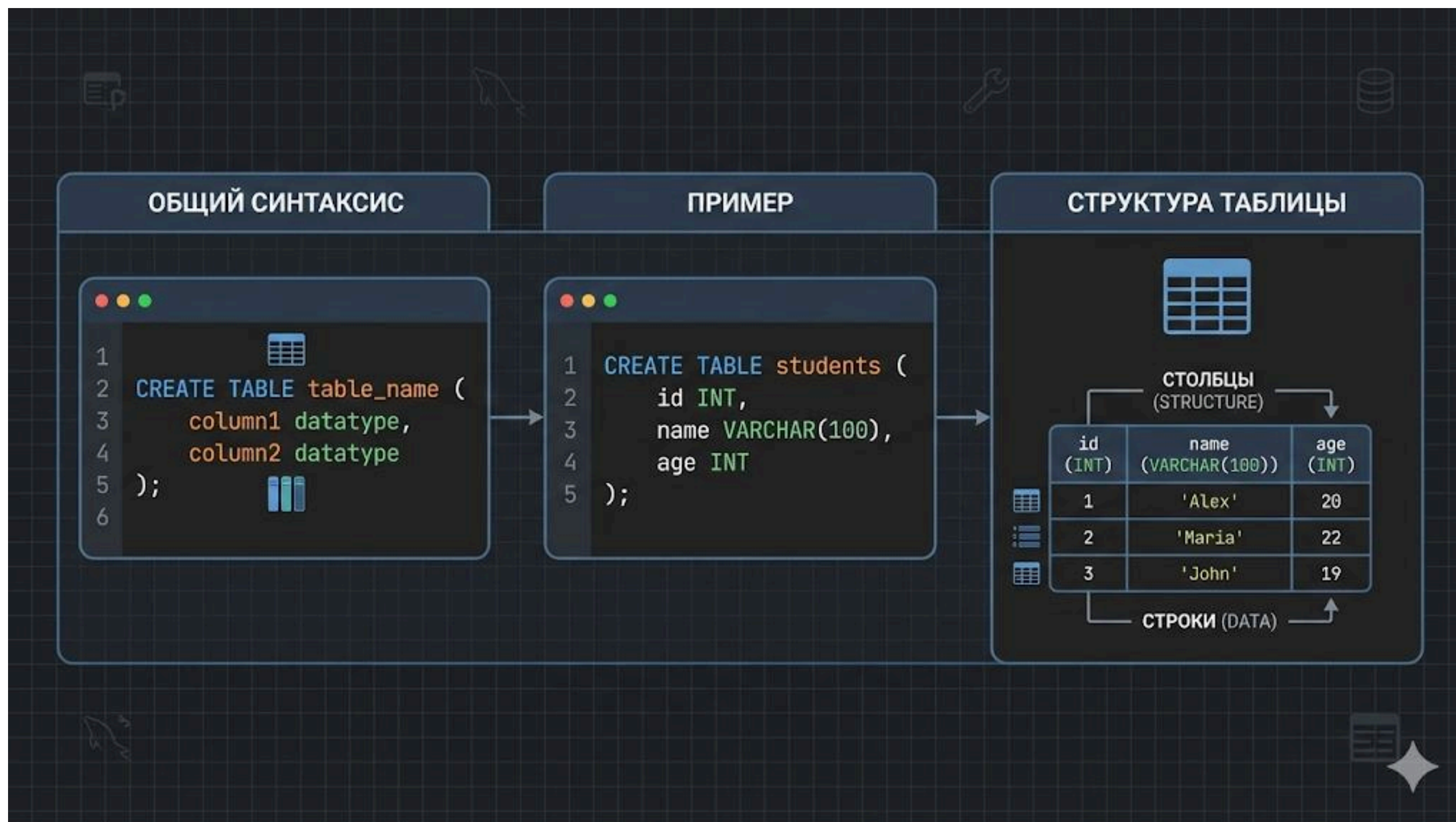
а. `CREATE DATABASE IF NOT EXISTS school;`

3. Выбор БД:

а. `USE school;`

CREATE TABLE — БАЗОВАЯ СТРУКТУРА

СИНТАКСИС СОЗДАНИЯ ТАБЛИЦЫ И ОПИСАНИЕ ПОЛЕЙ



1. Общий синтаксис:

а. CREATE TABLE table_name (
 column1 datatype,
 column2 datatype
);

2. Пример:

а. CREATE TABLE students (
 id INT,
 name VARCHAR(100),
 age INT
);

3. Таблица состоит из:

- столбцов (структура)
- строк (данные)

ПРИМЕР СОЗДАНИЯ ТАБЛИЦЫ PRODUCTS

ПОДРОБНЫЙ РАЗБОР СТРУКТУРЫ, ОГРАНИЧЕНИЙ И ИНДЕКСОВ

```
1 • CREATE TABLE products (  
2     product_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name VARCHAR(255) NOT NULL,  
4     product_price DECIMAL(10, 2) NOT NULL,  
5     product_description TEXT,  
6     product_img VARCHAR(500),  
7     category_id INT,  
8     stock INT DEFAULT 0,  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  
12    FOREIGN KEY (category_id)  
13        REFERENCES categories(category_id)  
14        ON DELETE SET NULL  
15        ON UPDATE CASCADE,  
16  
17    INDEX idx_category (category_id),  
18    INDEX idx_price (product_price),  
19    INDEX idx_active (is_active)  
20 ) ENGINE=InnoDB;
```

1. Общая структура

a. CREATE TABLE products (...)

ENGINE=InnoDB;

b. products — имя таблицы.

c. В круглых скобках описываются:

- поля
- ограничения
- внешние ключи
- индексы
- ENGINE=InnoDB — движок хранения.

2. Почему InnoDB?

- поддерживает FOREIGN KEY
- поддерживает транзакции
- обеспечивает ссылочную целостность

ПРИМЕР СОЗДАНИЯ ТАБЛИЦЫ PRODUCTS

ПОДРОБНЫЙ РАЗБОР СТРУКТУРЫ, ОГРАНИЧЕНИЙ И ИНДЕКСОВ

```
1 • CREATE TABLE products (  
2     product_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name VARCHAR(255) NOT NULL,  
4     product_price DECIMAL(10, 2) NOT NULL,  
5     product_description TEXT,  
6     product_img VARCHAR(500),  
7     category_id INT,  
8     stock INT DEFAULT 0,  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  
12    FOREIGN KEY (category_id)  
13        REFERENCES categories(category_id)  
14        ON DELETE SET NULL  
15        ON UPDATE CASCADE,  
16  
17    INDEX idx_category (category_id),  
18    INDEX idx_price (product_price),  
19    INDEX idx_active (is_active)  
20 ) ENGINE=InnoDB;
```

1. Первичный ключ

а. product_id INT AUTO_INCREMENT PRIMARY KEY

2. Разбор:

- INT — целое число
- AUTO_INCREMENT — автоматически увеличивается
- PRIMARY KEY — уникальный идентификатор строки

3. Каждый товар получает уникальный ID:

4. product_id product_name

5. 1	iPhone
2	Samsung

6. Остальные свойства указываются ниже.

ПРИМЕР СОЗДАНИЯ ТАБЛИЦЫ PRODUCTS

ПОДРОБНЫЙ РАЗБОР СТРУКТУРЫ, ОГРАНИЧЕНИЙ И ИНДЕКСОВ

```
1 • CREATE TABLE products (  
2     product_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name VARCHAR(255) NOT NULL,  
4     product_price DECIMAL(10, 2) NOT NULL,  
5     product_description TEXT,  
6     product_img VARCHAR(500),  
7     category_id INT,  
8     stock INT DEFAULT 0,  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  
12    FOREIGN KEY (category_id)  
13        REFERENCES categories(category_id)  
14        ON DELETE SET NULL  
15        ON UPDATE CASCADE,  
16  
17    INDEX idx_category (category_id),  
18    INDEX idx_price (product_price),  
19    INDEX idx_active (is_active)  
20 ) ENGINE=InnoDB;
```

1. Связь с категорией

а. category_id INT

б. Это поле будет связано с таблицей categories.

2. Внешний ключ

а. FOREIGN KEY (category_id)

REFERENCES categories(category_id)

ON DELETE SET NULL

ON UPDATE CASCADE

3. category_id в products ссылается на category_id в categories

4. ON DELETE SET NULL

а. Если категория удалена – у товара поле category_id станет NULL.

5. ON UPDATE CASCADE

а. Если ID категории изменится – автоматически обновится в products.

ПРИМЕР СОЗДАНИЯ ТАБЛИЦЫ PRODUCTS

ПОДРОБНЫЙ РАЗБОР СТРУКТУРЫ, ОГРАНИЧЕНИЙ И ИНДЕКСОВ

```
1 • CREATE TABLE products (  
2     product_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name VARCHAR(255) NOT NULL,  
4     product_price DECIMAL(10, 2) NOT NULL,  
5     product_description TEXT,  
6     product_img VARCHAR(500),  
7     category_id INT,  
8     stock INT DEFAULT 0,  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  
12    FOREIGN KEY (category_id)  
13        REFERENCES categories(category_id)  
14        ON DELETE SET NULL  
15        ON UPDATE CASCADE,  
16  
17    INDEX idx_category (category_id),  
18    INDEX idx_price (product_price),  
19    INDEX idx_active (is_active)  
20 ) ENGINE=InnoDB;
```

1. Индексы

- a. INDEX idx_category (category_id)
- b. INDEX idx_price (product_price)
- c. INDEX idx_active (is_active)

2. Индексы ускоряют поиск.

- a. SELECT * FROM products WHERE category_id = 5;
- b. SELECT * FROM products WHERE product_price < 1000;
- c. SELECT * FROM products WHERE is_active = TRUE;

3. Без индекса сервер будет сканировать всю таблицу.

4. С индексом поиск значительно быстрее.

ПРИМЕР СОЗДАНИЯ ТАБЛИЦЫ PRODUCTS

ПОДРОБНЫЙ РАЗБОР СТРУКТУРЫ, ОГРАНИЧЕНИЙ И ИНДЕКСОВ

```
1 • CREATE TABLE products (  
2     product_id INT AUTO_INCREMENT PRIMARY KEY,  
3     product_name VARCHAR(255) NOT NULL,  
4     product_price DECIMAL(10, 2) NOT NULL,  
5     product_description TEXT,  
6     product_img VARCHAR(500),  
7     category_id INT,  
8     stock INT DEFAULT 0,  
9     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
10    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
11  
12    FOREIGN KEY (category_id)  
13        REFERENCES categories(category_id)  
14        ON DELETE SET NULL  
15        ON UPDATE CASCADE,  
16  
17    INDEX idx_category (category_id),  
18    INDEX idx_price (product_price),  
19    INDEX idx_active (is_active)  
20 ) ENGINE=InnoDB;
```

1. Индексы

- a. INDEX idx_category (category_id)
- b. INDEX idx_price (product_price)
- c. INDEX idx_active (is_active)

2. Индексы ускоряют поиск.

- a. SELECT * FROM products WHERE category_id = 5;
- b. SELECT * FROM products WHERE product_price < 1000;
- c. SELECT * FROM products WHERE is_active = TRUE;

3. Без индекса сервер будет сканировать всю таблицу.

4. С индексом поиск значительно быстрее.

САМОСТОЯТЕЛЬНЫЕ УПРАЖНЕНИЯ

Блок 1. Базовые INSERT (Начальный уровень)

1. Добавление категории

Добавьте 5 новых категорий товаров с описанием.

2. Добавление пользователей

Добавьте:

- 3 обычных пользователя
- 1 администратора

3. Добавление товаров

Добавьте:

- минимум 10 товаров
- распределите их по категориям
- укажите stock
- укажите разные цены

4. Добавление заказа

Создайте заказ для существующего пользователя.

5. Добавление товаров в заказ

Добавьте 2-3 позиции в order_items для созданного заказа.

6. Добавление товара в корзину

Добавьте несколько товаров в корзину конкретного пользователя.

Блок 2. Простые SELECT

7. Получить всех пользователей

Вывести:

- id
- имя
- email
- роль

8. Получить активные товары

Вывести товары, где is_active = TRUE.

9. Найти товары дороже 5000

Использовать WHERE.

10. Найти товары по диапазону цены

Использовать BETWEEN.

11. Найти пользователей не администраторов

Использовать NOT.

12. Найти товары без категории

Использовать IS NULL.

Блок 3. UPDATE и DELETE

13. Обновить цену товара

Увеличить цену конкретного товара на 10%.

14. Изменить статус заказа

Перевести заказ из pending в processing.

15. Уменьшить stock товара после покупки

16. Удалить товар из корзины

Блок 4. Агрегатные функции

17. Посчитать количество пользователей

18. Посчитать общее количество товаров

19. Найти среднюю цену товаров

20. Найти самый дорогой и самый дешёвый товар

21. Посчитать общую сумму всех заказов

Блок 5. GROUP BY

22. Количество товаров в каждой категории

23. Количество заказов у каждого пользователя

24. Общая сумма заказов по каждому пользователю

25. Средняя стоимость заказа

САМОСТОЯТЕЛЬНЫЕ УПРАЖНЕНИЯ

Блок 6. HAVING

26. Пользователи с более чем 2 заказами

27. Пользователи, потратившие более 10 000

28. Категории, в которых больше 3 товаров

Блок 7. JOIN (Средний уровень)

29. Показать список заказов с именами пользователей

30. Показать товары вместе с названием категории

31. Показать корзину пользователя с названиями товаров

32. Показать детали конкретного заказа:

- имя пользователя

- название товара

- количество

- цена покупки

Блок 8. Сложные JOIN + GROUP BY

33. Посчитать сумму каждого заказа через order_items

(Сравнить с полем total)

34. Топ-5 самых продаваемых товаров

(по сумме quantity)

35. Общая выручка по категориям

36. Пользователь с максимальной суммой заказов

37. Количество проданных товаров по месяцам

Блок 9. Продвинутый уровень

38. Найти пользователей без заказов

(LEFT JOIN + IS NULL)

39. Найти товары, которые ни разу не покупали

40. Найти заказ с максимальной суммой

41. Рассчитать средний чек по всем заказам

42. Определить процент отменённых заказов

43. Найти повторные покупки одного и того же товара разными пользователями

44. Найти товары, которые есть в корзине, но отсутствуют в заказах

Блок 10. Сложная аналитика (для сильных студентов)

45. Построить отчёт:

Для каждого пользователя вывести:

- количество заказов

- общую сумму

- средний чек

- дату последнего заказа

46. Вывести категорию с максимальной выручкой

47. Найти 3 самых активных клиента по количеству заказов

48. Найти товар с самым большим суммарным оборотом (price * quantity)

49. Вывести список заказов со статусом shipped, где сумма выше средней по всем заказам

50. Построить полный отчёт по заказу:

- номер

- пользователь

- товары

- общая сумма

- статус

- дата



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?